

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**TIỂU LUẬN
MÔN PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG**

TÌM HIỂU VỀ FIREBASE FIRESTORE VÀ ỨNG DỤNG QUẢN LÝ SINH VIÊN

Người hướng dẫn: **ThS VŨ ĐÌNH HỒNG**

Người thực hiện: **NGUYỄN HÀ MY – 52200066**

MAI NGUYỄN PHƯƠNG TRANG – 52200051

ĐOÀN THỐNG LĨNH – 52200013

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2024

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**TIỂU LUẬN
MÔN PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG**

TÌM HIỂU VỀ FIREBASE FIRESTORE VÀ ỨNG DỤNG QUẢN LÝ SINH VIÊN

Người hướng dẫn: **ThS VŨ ĐÌNH HỒNG**

Người thực hiện: **NGUYỄN HÀ MY – 52200066**

MAI NGUYỄN PHƯƠNG TRANG – 52200051

ĐOÀN THỐNG LĨNH – 52200013

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2014

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến các thầy cô của trường Đại học Tôn Đức Thắng.

Trong những năm học tập và rèn luyện tại trường chúng em đã nhận được rất nhiều sự giúp đỡ từ thầy Vũ Đình Hồng. Thầy là người đã truyền lửa, truyền cảm hứng là tấm gương sáng cho chúng em noi theo. Thầy đã truyền cho chúng em những kiến thức bổ ích, những bài học cuộc sống, những câu chuyện ý nghĩa, không chỉ là kiến thức trong sách vở mà còn cả những bài học trong cuộc sống này. Với lòng biết ơn sâu sắc từ trong đáy lòng của chúng em, chúng em xin gửi lời cảm ơn sâu sắc đến các thầy cô đã dẫn dắt em trong môn Phát triển ứng dụng di động.

Trong quá trình thực hiện tiểu luận, em khó có thể tránh khỏi những sai sót, rất mong quý thầy cô có thể đóng góp ý kiến để em có được thêm nhiều bài học và rút kinh nghiệm cho lần sau.

ĐỒ ÁN ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là sản phẩm đồ án của riêng chúng tôi và được sự hướng dẫn của thầy Vũ Đình Hồng;. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong đồ án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung đồ án của mình. Trường đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày tháng năm 2024

Tác giả

(ký tên và ghi rõ họ tên)

Mai Nguyễn Phương Trang

Đoàn Thống Lĩnh

Nguyễn Hà My

PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN

Phần xác nhận của GV hướng dẫn

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

Phần đánh giá của GV chấm bài

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

TÓM TẮT

Bài tiểu luận này nghiên cứu toàn diện về Firebase và đặc biệt là Google Firebase Firestore để phát triển ứng dụng thời gian thực cho mục tiêu quản lý thông tin sinh viên. Dự án này nhằm mục đích có được sự hiểu biết thấu đáo về Firebase Firestore để phát triển một ứng dụng cung cấp các chức năng khác nhau liên quan đến quản lý thông tin sinh viên. Chức năng phần mềm chính chia làm 3 phần:

- User Account Management: bao gồm các chức năng đăng nhập, cập nhật ảnh người dùng, xem danh sách người dùng, thêm người dùng mới, xóa người dùng, chỉnh sửa thông tin và xem lịch sử đăng nhập của user.
- Student Management: bao gồm các chức năng xem danh sách sinh viên, thêm sinh viên mới, xóa sinh viên, sửa thông tin sinh viên, sắp xếp danh sách sinh viên dựa trên nhiều tiêu chí, tìm kiếm sinh viên bằng nhiều tiêu chí, truy cập trang cá nhân của sinh viên, xem thông tin sinh viên trên một màn hình chi tiết, quản lý danh sách chứng chỉ của sinh viên, xem danh sách chứng chỉ, xem thông tin chứng chỉ, sửa thông tin chứng chỉ, xóa chứng chỉ.
- Data Import/Export: Import danh sách sinh viên từ một tệp, xuất danh sách sinh viên sang Excel/CSV, nhập danh sách chứng chỉ cho học sinh từ tệp, xuất danh sách chứng chỉ sang Excel/CSV.

MỤC LỤC

LỜI CẢM ƠN	i
ĐỒ ÁN ĐƯỢC HOÀN THÀNH	ii
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG.....	ii
PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN	iii
TÓM TẮT	iv
MỤC LỤC.....	1
DANH MỤC KÍ HIỆU VÀ CHỮ VIẾT TẮT	3
DANH MỤC CÁC BẢNG BIỂU, HÌNH VẼ, ĐỒ THỊ	4
CHƯƠNG 1 – GIỚI THIỆU FIREBASE	6
1.1 Tổng quan về Firebase	6
1.1.1 Giới thiệu về Firebase.....	6
1.1.2 So sánh cách hoạt động của Firebase với cách truyền thống	7
1.2 Một số tính năng của Firebase	9
1.3 Một số dịch vụ mà Firebase cung cấp.....	12
1.4 Ưu và nhược điểm của Firebase	20
1.4.1 Ưu điểm của Firebase	20
1.4.2 Nhược điểm của Firebase	20
CHƯƠNG 2 – LÝ THUYẾT VỀ FIREBASE FIRESTORE	22
2.1 Kiến trúc của Firebase Firestore	22
2.2 Cơ chế hoạt động và một số tính năng của Firebase Firestore	24
2.3 Truy vấn và lọc dữ liệu trong Firestore	26
2.4 Quy tắc bảo mật trong Firebase Firestore	27
2.5 Ưu và nhược điểm của Firebase Firestore	28
CHƯƠNG 3 – TỔNG QUAN VỀ DỰ ÁN.....	30
3.1 Giới thiệu về dự án.....	30
3.2 Đối tượng người dùng của dự án	30

3.3	Công nghệ sử dụng trong dự án và lý do sử dụng	30
CHƯƠNG 4 – KIẾN TRÚC VÀ TRIỂN KHAI DỰ ÁN.....		33
4.1	Kiến trúc tổng thể của hệ thống	33
4.2	Thiết kế cơ sở dữ liệu.....	34
4.3	Thiết kế giao diện.....	36
4.4	Quy trình triển khai	39
4.4.1	Tích hợp firebase vào ứng dụng	39
4.4.2	Hiện thực ứng dụng	42

DANH MỤC KÍ HIỆU VÀ CHỮ VIẾT TẮT

CÁC KÝ HIỆU

CÁC CHỮ VIẾT TẮT

DANH MỤC CÁC BẢNG BIỂU, HÌNH VẼ, ĐỒ THỊ

DANH MỤC HÌNH

Hình 1: So sánh sử dụng firebase với cách truyền thống.....	7
Hình 2: Một số tính năng của firebase	9
Hình 3: Firebase Realtime Database	13
Hình 4: So sánh Realtime Database và Cloud Firestore	14
Hình 5: Firebase Authentication	15
Hình 6: Firebase Hosting	16
Hình 7: Kiến trúc Firsebase Hosting.....	16
Hình 8: Firebase Cloud Message	17
Hình 9: Kiến trúc của Firebase Cloud Message.....	17
Hình 10: Firebase Cloud Functions	18
Hình 11: Firebase Analytics.....	19
Hình 12: Firebase Cloud Storage	20
Hình 13: Firebase Firestore.....	22
Hình 14: Kiến trúc của Firestore.....	23
Hình 15: Ví dụ về 1 collection trong Firebase Firestore.....	23
Hình 16: Ví dụ về lọc dữ liệu trong firebase.....	26
Hình 17: Ví dụ về lọc dữ liệu trong Firestore	27
Hình 18: Ví dụ về quy tắc bảo mật cơ bản trong firestore	28
Hình 19: Kiến trúc tổng thể của hệ thống	33
Hình 20: Màn hình login	36
Hình 21: Màn hình trang chủ quản lý tài khoản người dùng	37
Hình 22: Màn hình xem chi tiết, thêm mới và sửa thông tin người dùng.....	38
Hình 23: Màn quản lý, xem chi tiết, thêm mới và sửa thông tin sinh viên.....	38
Hình 24: Màn hình quản lý, xem chi tiết, thêm mới và sửa thông tin chứng chỉ	39
Hình 25: Quy trình tích hợp firebase vào ứng dụng	39

Hình 26: Quy trình tích hợp firebase vào ứng dụng	40
Hình 27: Quy trình tích hợp firebase vào ứng dụng	40
Hình 28: Quy trình tích hợp firebase vào ứng dụng	41
Hình 29: Quy trình tích hợp firebase vào ứng dụng	41
Hình 30: Màn hình Login.....	42
Hình 31: Validate màn hình Login.....	43
Hình 32: Màn hình quản lý tài khoản người dùng	44
Hình 33: Màn hình xem và sửa thông tin tài khoản người dùng	45
Hình 34: Màn hình thêm và xóa người dùng	46
Hình 35: Màn hình quản lý student và chức năng tìm kiếm	47
Hình 36: Chức năng lọc sinh viên theo nhiều tiêu chí	48
Hình 37: Chức năng lọc sinh viên theo tên và ID	49
Hình 38: Màn hình thêm student và xóa student	50
Hình 39: Màn hình xem thông tin chi tiết của student.....	51
Hình 40: Màn hình chỉnh sửa thông tin chi tiết của student	52
Hình 41: Màn hình xem thông tin chứng chỉ của student.....	53
Hình 42: Màn hình thêm chứng chỉ cho sinh viên.....	54
Hình 43: Màn hình xem và sửa thông tin certificate chi tiết của student	55
Hình 44: Màn hình xem lịch sử đăng nhập	56

DANH MỤC BẢNG

Bảng 1: So sánh phát triển ứng dụng bằng Firebase và với phương thức truyền thống ..	8
Bảng 2: Các tính năng của Firebase	12
Bảng 3: So sánh Realtime Database và Firebase Firestore	26
Bảng 4: Ưu và nhược điểm của Firestore	29

CHƯƠNG 1 – GIỚI THIỆU FIREBASE

1.1 Tổng quan về Firebase

1.1.1 Giới thiệu về Firebase

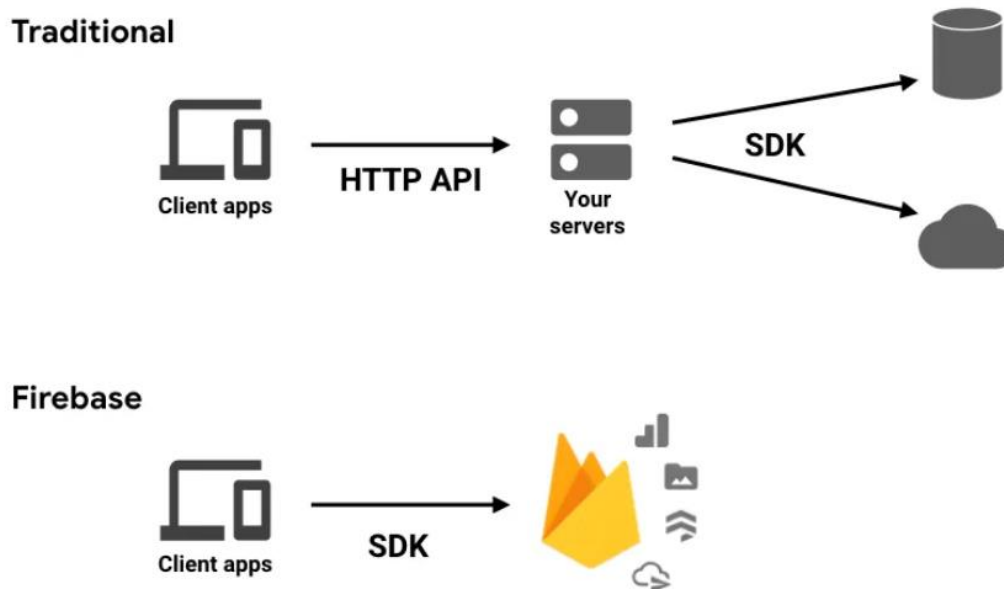
Firebase là một nền tảng được phát triển bởi Google, cung cấp nhiều dịch vụ hỗ trợ xây dựng ứng dụng, đặc biệt phù hợp với các ứng dụng di động và web, bao gồm các API đơn giản và mạnh mẽ mà không cần backend hay server. Ban đầu, Firebase ra đời với mục tiêu hỗ trợ các nhà phát triển dễ dàng quản lý dữ liệu theo thời gian thực. Sau khi Google mua lại Firebase vào năm 2014, nó đã được mở rộng thành một bộ công cụ phong phú, mạnh đến cho nhà phát triển khả năng triển khai, quản lý, và cải tiến ứng dụng của họ một cách toàn diện.

Firebase hoạt động dựa trên nền tảng đám mây của Google và tận dụng sức mạnh của các máy chủ hiệu suất cao, giúp các lập trình viên thao tác với cơ sở dữ liệu một cách đơn giản hơn. Bằng cách cung cấp các API dễ sử dụng, Firebase cho phép các nhà phát triển tích hợp các tính năng vào ứng dụng của mình với quy trình tinh gọn, giúp tối ưu hóa trải nghiệm người dùng và nâng cao lợi nhuận.

Đặc biệt, firebase còn là một nền tảng đa năng và bảo mật cực tốt, hỗ trợ cả hai hệ điều hành Android và IOS. Đây là lý do khiến Firebase được rất nhiều lập trình viên tin dùng khi phát triển các ứng dụng phục vụ số lượng lớn người dùng trên toàn cầu.

Firebase mang đến một giải pháp **Backend as a Service (BaaS)**, giúp các nhà phát triển tránh phải tự xây dựng các hệ thống phức tạp nhưng vẫn có thể quản lý cơ sở dữ liệu, xác thực người dùng, lưu trữ tệp tin, và nhiều tính năng khác. Firebase cung cấp sẵn các dịch vụ như lưu trữ thời gian thực, thông báo đẩy, phân tích dữ liệu, và xác thực người dùng. Điều này cho phép các nhà phát triển dễ dàng bổ sung các tính năng cần thiết vào ứng dụng mà không cần thiết lập backend từ đầu.

1.1.2 So sánh cách hoạt động của Firebase với cách truyền thống



Hình 1: So sánh sử dụng firebase với cách truyền thống

Firebase hoạt động bằng cách cung cấp các SDK cho phía client, cho phép ứng dụng trực tiếp sử dụng với các dịch vụ backend mà không cần thiết lập bất kỳ middleware nào giữa ứng dụng và dịch vụ.

Những Client SDK này do Firebase cung cấp cho phép ứng dụng dễ dàng truy cập và tương tác trực tiếp với các dịch vụ backend của Firebase từ phía client. Điều này nghĩa là các thao tác như truy vấn cơ sở dữ liệu hay gọi các dịch vụ khác có thể được thực hiện trực tiếp trong mã của ứng dụng mà không cần thông qua bất kỳ lớp xử lý trung gian nào.

Trong các mô hình phát triển ứng dụng truyền thống, bạn thường phải xây dựng mã cho cả frontend và backend. Frontend sẽ gửi yêu cầu tới các API endpoint được thiết lập bởi backend, còn backend sẽ chịu trách nhiệm xử lý các tác vụ quan trọng như truy vấn dữ liệu, xác thực, và xử lý các logic nghiệp vụ. Tuy nhiên, với các sản phẩm của Firebase, phần backend truyền thống được bỏ qua, và phần lớn tác vụ có thể được xử lý ngay từ phía client. Điều này giúp quy trình phát triển trở nên đơn giản và nhanh chóng

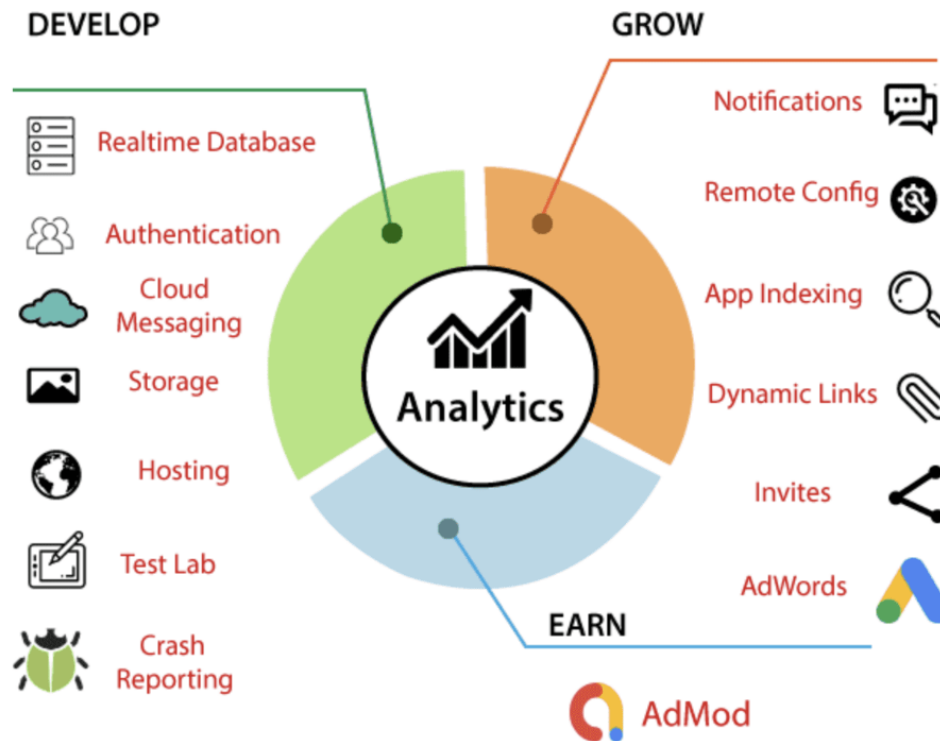
hơn, đặc biệt hữu ích khi phát triển các ứng dụng có quy mô nhỏ hoặc cần xây dựng các prototype nhanh chóng.

Bảng so sánh phát triển ứng dụng bằng Firebase và với phương thức truyền thống

Yếu tố	Firebase	Phương pháp truyền thống
Cơ sở hạ tầng backend	Hoạt động theo mô hình không máy chủ (Serverless)	Máy chủ tự thiết lập và quản lý
Cập nhật dữ liệu	Tự động đồng bộ dữ liệu thời gian thực	Yêu cầu làm mới và kiểm tra dữ liệu thủ công
Xác thực người dùng	Tích hợp sẵn các công cụ xác thực của Firebase	Triển khai hệ thống xác thực riêng hoặc sử dụng thư viện ngoài
Lưu trữ dữ liệu	Sử dụng dịch vụ Firebase Cloud Storage	Lưu trữ qua hệ thống máy chủ và cơ sở hạ tầng tự quản lý
Cơ sở dữ liệu	Cung cấp Firebase Realtime Database và Cloud Firestore	Cơ sở dữ liệu được thiết lập và quản lý thủ công
Khả năng mở rộng	Tự động mở rộng quy mô và quản lý tài nguyên	Quy trình mở rộng và cung cấp tài nguyên cần thực hiện thủ công
Bảo trì và cập nhật	Firebase tự động xử lý bảo trì và cập nhật hệ thống	Người dùng tự quản lý bảo trì và cập nhật máy chủ
Tốc độ phát triển	Phát triển ứng dụng nhanh chóng với các dịch vụ có sẵn	Thời gian phát triển kéo dài do cần xây dựng phần back-end riêng

Bảng 1: So sánh phát triển ứng dụng bằng Firebase và với phương thức truyền thống

1.2 Một số tính năng của Firebase



Hình 2: Một số tính năng của firebase

Firebase cung cấp một bộ tính năng toàn diện để hỗ trợ các nhà phát triển trong việc xây dựng, phát hành, giám sát, và tương tác với ứng dụng của họ. Các tính năng này được chia thành ba nhóm chính: Build (xây dựng), Release & Monitor (phát hành và giám sát), và Engage (tương tác).

- **Firestore Build Features:** Nhóm tính năng này tập trung vào việc cung cấp các công cụ cốt lõi để phát triển ứng dụng như cơ sở dữ liệu, xác thực và lưu trữ.
 - **Databases:** Firebase cung cấp hai giải pháp lưu trữ dữ liệu đám mây là Cloud Firestore và Realtime Database, giúp đồng bộ hóa dữ liệu nhanh chóng và hiệu quả.
 - **Machine Learning:** Firebase ML hỗ trợ tích hợp các khả năng học máy vào ứng dụng, giúp nâng cao trải nghiệm người dùng mà không cần kiến thức chuyên sâu về học máy.

- Cloud Functions: Đây là dịch vụ serverless cho phép viết mã backend để phản hồi các sự kiện hoặc yêu cầu HTTP, giúp giảm tải cho ứng dụng client.
 - Authentication: Firebase Authentication cung cấp hệ thống xác thực dễ sử dụng, tích hợp nhiều phương thức đăng nhập qua Google, Facebook, Twitter và các nền tảng phổ biến khác.
 - Cloud Messaging: Firebase Cloud Messaging (FCM) là một dịch vụ hỗ trợ gửi thông báo và tin nhắn dữ liệu miễn phí, hỗ trợ đa nền tảng.
 - Hosting: Firebase Hosting cung cấp giải pháp lưu trữ có thể mở rộng cho các ứng dụng web và dịch vụ microservices.
 - Cloud Storage: Dịch vụ này cho phép lưu trữ và quản lý tài nguyên ứng dụng và nội dung do người dùng tạo ra một cách an toàn.
 - Emulator Suite: Bộ Local Emulator Suite cho phép thử nghiệm tính năng mà không phát sinh thêm chi phí sử dụng.
- **Firestore Release & Monitor Features:** Nhóm này hỗ trợ giám sát và phát hiện lỗi trong quá trình vận hành nhằm đảm bảo hiệu suất ổn định cho ứng dụng.
- Crashlytics: Firebase Crashlytics là một công cụ báo cáo lỗi theo thời gian thực, giúp các nhà phát triển nhanh chóng phát hiện và xử lý vấn đề về độ ổn định.
 - Analytics: Google Analytics được tích hợp với Firebase, cung cấp cái nhìn sâu sắc về hành vi người dùng và hiệu suất ứng dụng.
 - Performance Monitoring: Công cụ theo dõi hiệu suất giúp đo lường và phân tích các khía cạnh hiệu suất của ứng dụng trên iOS, Android và web.
 - Test Lab: Firebase Test Lab là một cơ sở hạ tầng đám mây hỗ trợ kiểm tra, thử nghiệm ứng dụng trên nhiều thiết bị và cấu hình khác nhau.

- App Distribution: Tính năng này hỗ trợ quá trình thử nghiệm beta, cho phép phân phối ứng dụng dễ dàng tới người dùng thử nghiệm trước khi phát hành chính thức.
- **Firestore Engage Features:** Nhóm tính năng này tập trung vào việc cải thiện tương tác và tối ưu hóa trải nghiệm người dùng.
 - Remote Config: Firebase Remote Config cho phép các nhà phát triển thay đổi hành vi và giao diện ứng dụng mà không cần cập nhật ứng dụng.
 - Predictions: Tính năng này sử dụng học máy để tạo các phân đoạn người dùng dựa trên dự đoán hành vi.
 - A/B Testing: Firebase A/B Testing giúp tối ưu hóa trải nghiệm ứng dụng bằng cách thử nghiệm các biến thể khác nhau của giao diện người dùng, tính năng và chiến dịch.
 - Dynamic Links: Tính năng Dynamic Links tạo ra các liên kết thông minh dẫn đến nội dung cụ thể trong ứng dụng, bất kể ứng dụng đã được cài đặt hay chưa.
 - In-App Messaging: Tính năng này cho phép gửi các tin nhắn có mục tiêu và ngữ cảnh đến người dùng đang sử dụng ứng dụng.
- ❖ Nhìn chung, Firebase mang lại một bộ công cụ phong phú và mạnh mẽ giúp các nhà phát triển dễ dàng triển khai, xây dựng, phát triển và quản lý ứng dụng tối ưu hóa các hoạt động phát triển và cải thiện trải nghiệm người dùng.

Các tính năng chính của Firebase được phân loại theo 3 nhóm như trên tương ứng với 3 giai đoạn trong chu kỳ phát triển ứng dụng:

Giai đoạn	Lợi ích chính của Firebase	Các tính năng của Firebase
Build (Xây dựng)	Giúp xây dựng ứng dụng nhanh chóng	• Cơ sở dữ liệu (Firestore + Realtime Database)

		• Firebase Machine Learning (FML) • Cloud Functions • Authentication • Cloud Messaging • Hosting • Cloud Storage Emulator Suite
Release & Monitor (Phát hành & Giám sát)	Cung cấp các công cụ kiểm tra, phân tích và phân phối, đảm bảo trải nghiệm ứng dụng mượt mà cho người dùng	• Crashlytics • Analytics • Performance Monitoring • Test Lab • App Distribution
Grow & Engage (Phát triển lớn mạnh & Tương tác)	Cung cấp công cụ nâng cao để tăng cường quá trình tương tác của người dùng, giúp tiếp cận và thu hút khách hàng	• Remote Config • Predictions • A/B Testing • Dynamic link • In-app messaging

Bảng 2: Các tính năng của Firebase

1.3 Một số dịch vụ mà Firebase cung cấp

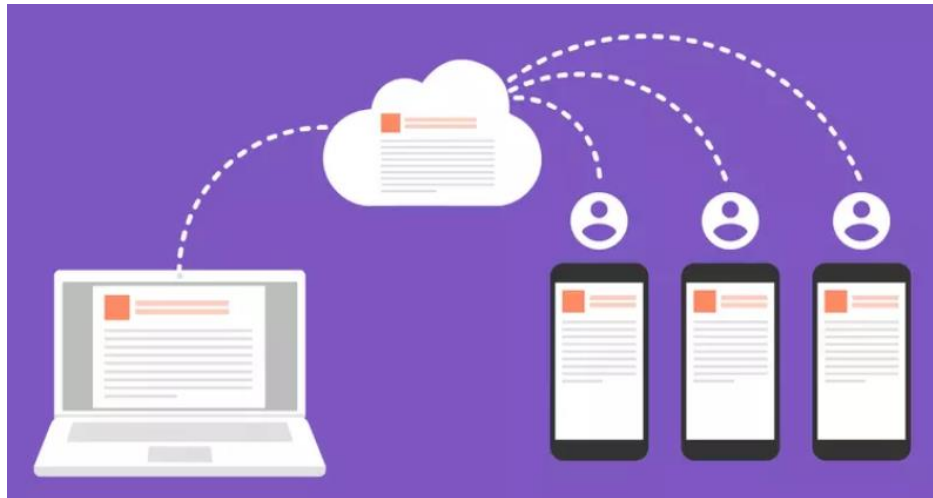
Firebase cung cấp một bộ dịch vụ đa dạng hỗ trợ các nhà phát triển từ backend, phân tích, phát triển giao diện đến tối ưu hóa trải nghiệm người dùng. Dưới đây là các dịch vụ nổi bật của Firebase:

- **Firebase Realtime Database:** Firebase Realtime Database là cơ sở dữ liệu NoSQL theo thời gian thực, sử dụng định dạng JSON để lưu trữ và đồng bộ hóa dữ liệu ngay lập tức giữa tất cả các client khi có bất kỳ thay đổi nào. Điều này rất lý tưởng cho các

ứng dụng chat, mạng xã hội, hoặc những ứng dụng yêu cầu cập nhật dữ liệu liên tục mà không cần phải tải lại trang.

Đặc biệt, trong các ứng dụng đa nền tảng, tất cả các client sẽ đồng bộ với cùng một cơ sở dữ liệu, đảm bảo rằng dữ liệu được cập nhật liên tục và chính xác trên tất cả các thiết bị khi có sự thay đổi. Quá trình này được thực hiện thông qua kết nối bảo mật SSL với chứng chỉ 2048 bit, đảm bảo an toàn cho dữ liệu khi truyền tải.

Trong trường hợp mất kết nối mạng, Firebase Realtime Database sẽ lưu trữ tạm thời dữ liệu ở chế độ offline trên thiết bị. Khi kết nối mạng được khôi phục, tất cả thay đổi sẽ được tự động đồng bộ và cập nhật lên server của Firebase. Hệ thống cũng tự động đồng bộ hóa dữ liệu local cũ hơn với server để đảm bảo rằng người dùng luôn có dữ liệu mới nhất khi kết nối lại.



Hình 3: Firebase Realtime Database

- **Firebase Cloud Firestore:** Firebase Cloud Firestore là một hệ thống cơ sở dữ liệu được phát triển từ Firebase Realtime Database, mang đến nhiều cải tiến về giao diện và khả năng mở rộng. So với Realtime Database, Cloud Firestore có khả năng xử lý dữ liệu tốt hơn, với các tính năng nâng cao giúp đồng bộ hóa dữ liệu trên các ứng dụng thông qua việc đăng ký thời gian thực và cung cấp hỗ trợ ngoại tuyến cho cả thiết bị di động lẫn các ứng dụng web.

Cloud Firestore là cơ sở dữ liệu NoSQL với cấu trúc linh hoạt, sử dụng mô hình document và collection. Điều này giúp Firestore hỗ trợ các truy vấn mạnh mẽ hơn, cho phép thực hiện các truy vấn phức tạp và đồng bộ hóa dữ liệu theo thời gian thực.

Với các tính năng mạnh mẽ và khả năng mở rộng, Cloud Firestore thường được sử dụng trong các ứng dụng phức tạp, nơi cần lưu trữ và truy vấn dữ liệu chi tiết hơn. Nó có thể thay thế các cơ sở dữ liệu truyền thống khi xây dựng ứng dụng, mang lại sự linh hoạt và hiệu quả cao trong việc quản lý và truy xuất dữ liệu.

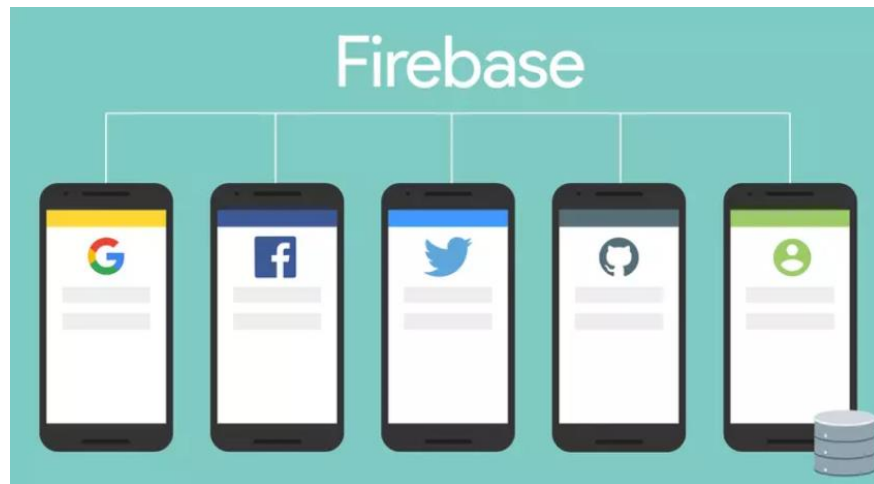
 Realtime Database	 Cloud Firestore
 Allows single sorting condition.	 Allows multiple field comparison.
 Only hosted in North America.	 Allows multi regional support.
 Transfer occurs via JSON files.	 Fetching query is organic.

Hình 4: So sánh Realtime Database và Cloud Firestore

- **Firebase Authentication:** Firebase Authentication là dịch vụ của Firebase cung cấp các tính năng xác thực người dùng, giúp việc đăng ký, đăng nhập và quản lý tài khoản người dùng trở nên đơn giản trong các ứng dụng di động và web. Dịch vụ này hỗ trợ nhiều phương thức xác thực khác nhau, bao gồm đăng nhập qua email và mật khẩu, mạng xã hội (Facebook, Google, Twitter), xác thực qua vân tay, và xác thực số điện thoại, đáp ứng nhu cầu đa dạng của người dùng.

Với Firebase Authentication, bạn có thể dễ dàng tích hợp các phương thức xác thực này vào ứng dụng của mình, giúp người dùng đăng ký và đăng nhập nhanh chóng bằng nhiều cách khác nhau. Hệ thống cũng cung cấp các API mạnh mẽ cho phép bạn quản lý người dùng, bao gồm các chức năng tạo mới, xóa và cập nhật thông tin người dùng như tên, ảnh đại diện và các thông tin khác.

Đặc biệt, Firebase Authentication hỗ trợ các thư viện xác thực cho các nền tảng phổ biến như Android, iOS và web, giúp bạn dễ dàng tích hợp vào ứng dụng và quản lý quy trình xác thực một cách hiệu quả..

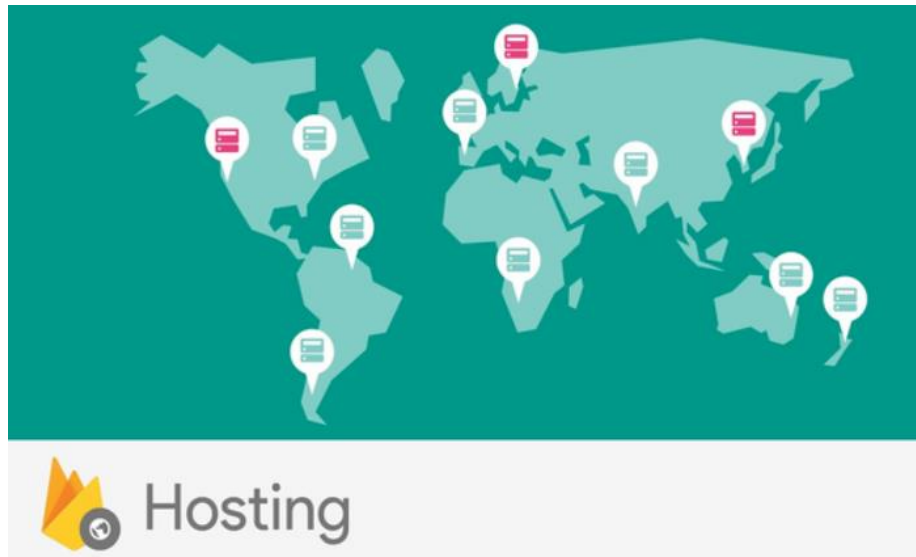


Hình 5: Firebase Authentication

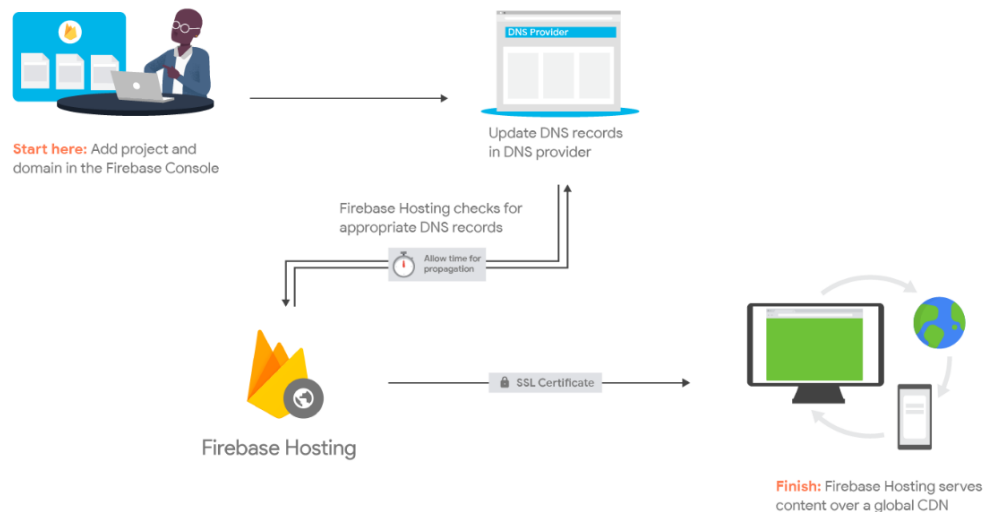
- **Firebase Hosting:** Firebase Hosting là dịch vụ lưu trữ web mạnh mẽ, an toàn và nhanh chóng, lý tưởng cho các ứng dụng web tĩnh (HTML, CSS, JavaScript) hoặc ứng dụng động. Dịch vụ này đặc biệt phù hợp với các trang landing page, ứng dụng đơn trang (single-page applications) và Progressive Web Apps (PWA) nhờ vào khả năng tải nhanh và mức độ bảo mật cao. Firebase Hosting hỗ trợ HTTPS mặc định, tích hợp mạng phân phối nội dung CDN, và cung cấp phương thức triển khai thuận tiện thông qua CLI.

Mạng phân phối nội dung (CDN - Content Delivery Network) là một hệ thống các máy chủ phân phối bản sao nội dung tĩnh của website đến nhiều điểm trên toàn

cầu, gọi là Points of Presence (PoP). Khi người dùng truy cập website, dữ liệu được gửi từ máy chủ gần nhất, giúp giảm độ trễ và tăng tốc độ tải trang. Nhờ CDN, người dùng có thể truy cập bản sao của nội dung từ các máy chủ gần họ nhất, mang lại trải nghiệm mượt mà và nhanh chóng.



Hình 6: Firebase Hosting



Hình 7: Kiến trúc Firsebase Hosting

- **Firebase Cloud Message:** Firebase Cloud Messaging (FCM) là một tính năng quan trọng của nền tảng Firebase, cho phép các nhà phát triển xây dựng các ứng dụng chat

và gửi thông báo đến nhiều thiết bị khác nhau, bao gồm web, Android và iOS. FCM cho phép chủ sở hữu ứng dụng gửi tin nhắn (tối đa 4KB đối với thông báo tức thời) nhằm thu hút và giữ người dùng, tạo cơ hội giao tiếp hiệu quả với người dùng ngay cả khi họ không sử dụng ứng dụng.

Với FCM, các nhà phát triển có thể gửi thông báo đẩy (push notifications), cập nhật nội dung ứng dụng và nhiều tác vụ khác mà không yêu cầu người dùng phải tương tác trực tiếp. Một điểm đáng chú ý là dịch vụ này không yêu cầu mã hóa phức tạp, giúp dễ dàng triển khai. FCM tích hợp chặt chẽ với Firebase Analytics, mang đến khả năng theo dõi chi tiết mức độ tương tác và theo dõi chuyển đổi, giúp phân tích hành vi người dùng.

FCM còn cung cấp tùy chọn gửi thông báo ngay lập tức hoặc theo lịch trình tại một thời điểm cụ thể trong tương lai, dựa trên múi giờ địa phương của người nhận, cùng nhiều tùy chỉnh khác mà bạn có thể khám phá thêm.



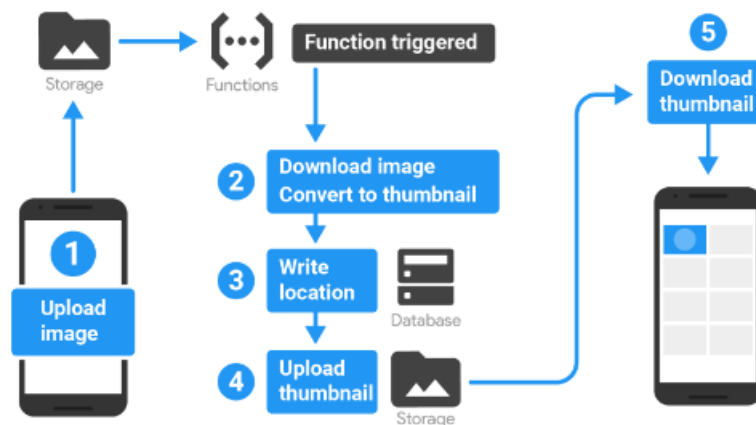
Hình 8: Firebase Cloud Message



Hình 9: Kiến trúc của Firebase Cloud Message

- **Firestore Cloud Functions:** Firestore Cloud Functions là một nền tảng dựa trên điện toán đám mây không có máy chủ (serverless), cho phép developer chạy code backend tùy chỉnh để phản hồi các sự kiện. Giúp xây dựng các tính năng backend mà không cần duy trì máy chủ, ví dụ xử lý thanh toán, gửi thông báo đẩy, và bảo trì cơ sở dữ liệu.

Developer có thể viết các hàm phản hồi các thay đổi cơ sở dữ liệu, hành động của người dùng, yêu cầu HTTP,... và có thể được thực thi trong một môi trường được quản lý, giúp giảm nguồn lực dành cho việc mở rộng quy mô và quản lý máy chủ.

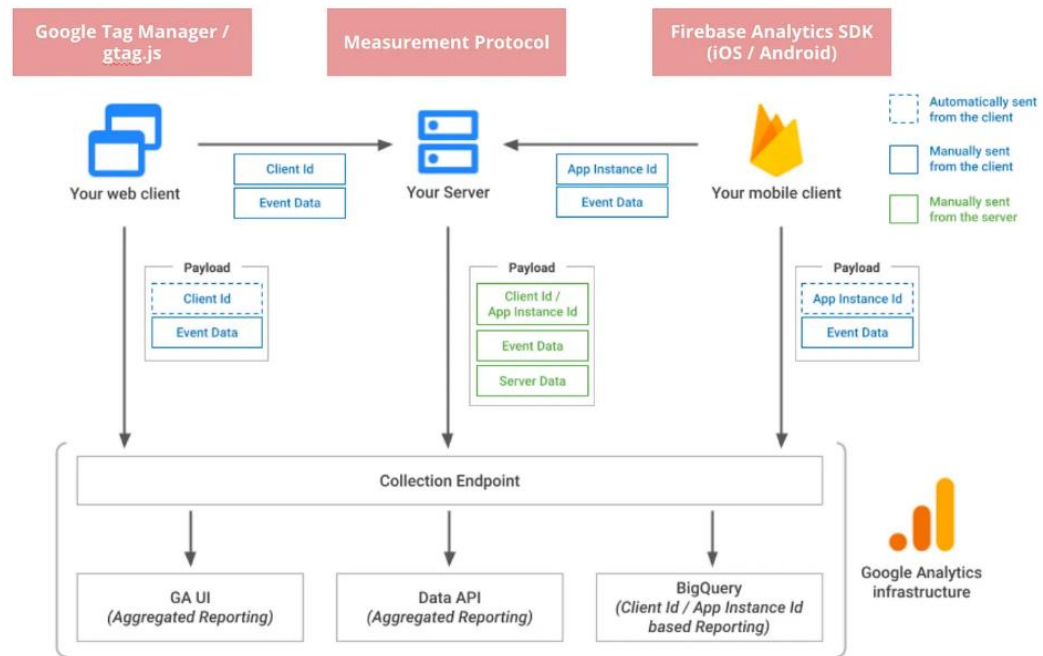


Hình 10: Firestore Cloud Functions

- **Firestore Analytics:** Firestore Analytics là công cụ giúp phân tích hành vi người dùng trong ứng dụng, cung cấp thông tin chi tiết về các sự kiện và các chỉ số quan trọng như số lượng người dùng, thời gian sử dụng ứng dụng và hành vi của người dùng. Công cụ này rất hữu ích trong việc tối ưu hóa trải nghiệm người dùng và các chiến lược kinh doanh, ví dụ: giúp bạn hiểu rõ các tính năng nào được người dùng yêu thích và sử dụng nhiều nhất, từ đó điều chỉnh ứng dụng cho phù hợp với nhu cầu của họ.

Thông qua Firestore Analytics, bạn có thể biết được các tính năng mà khách hàng thường xuyên truy cập, cùng với các thông tin quan trọng như hiệu quả của các chiến dịch quảng cáo hay tình hình trả phí, từ đó xây dựng chiến lược phát triển ứng

dụng hợp lý. Để sử dụng tính năng Analytics, bạn cần cài đặt phần mềm SDK. Firebase Analytics còn tích hợp với các dịch vụ khác như FCM và Remote Config, giúp các nhà phát triển tối ưu hóa trải nghiệm người dùng và thực hiện các chiến dịch marketing hiệu quả hơn.



Hình 11: Firebase Analytics

- **Firebase Cloud Storage:** Cloud Storage cung cấp bộ nhớ đám mây an toàn, có thể mở rộng, là tính năng cho phép lưu trữ và quản lý nội dung đã tạo ra như ảnh, video, nội dung, văn bản,... Firebase Storage cung cấp các API hỗ trợ bạn upload và download các file từ ứng dụng một cách trơn tru mà không cần quan tâm đến chất lượng đường truyền mạng với độ bảo mật cao.



Hình 12: Firebase Cloud Storage

1.4 Ưu và nhược điểm của Firebase

1.4.1 Ưu điểm của Firebase

- Tạo tài khoản và sử dụng miễn phí:
- Tốc độ phát triển nhanh và tiết kiệm thời gian
- Hỗ trợ nhiều dịch vụ trong một nền tảng
- Được cung cấp bởi Google
- Phát triển giao diện người dùng
- Cung cấp môi trường không có máy chủ
- Sử dụng Học máy (Machine Learning)
- Sao lưu thường xuyên
- Hỗ trợ lập chỉ mục
- Tính năng Crashlytics

1.4.2 Nhược điểm của Firebase

- Thuộc mã nguồn đóng
- Người dùng không có quyền truy cập vào mã nguồn
- Bị chặn và không truy cập ở nhiều quốc gia
- Chỉ hoạt động trên cơ sở dữ liệu NoSQL
- Tốc độ truy vấn chậm

- Không phải tất cả dịch vụ Firebase đều miễn phí
- Chi phí khá đắt và không ổn định
- Chỉ hoạt động trên Google Cloud
- Không có Dedicated Servers và hợp đồng doanh nghiệp
- Không cung cấp các API GraphQL

CHƯƠNG 2 – LÝ THUYẾT VỀ FIREBASE FIRESTORE

2.1 Kiến trúc của Firebase Firestore



Hình 13: Firebase Firestore

Cloud Firestore là một cơ sở dữ liệu NoSQL, hướng đến tài liệu. Không giống như cơ sở dữ liệu SQL, cơ sở dữ liệu NoSQL không có bảng hoặc hàng. Thay vào đó, nó lưu trữ dữ liệu trong tài liệu, được sắp xếp thành bộ sưu tập.

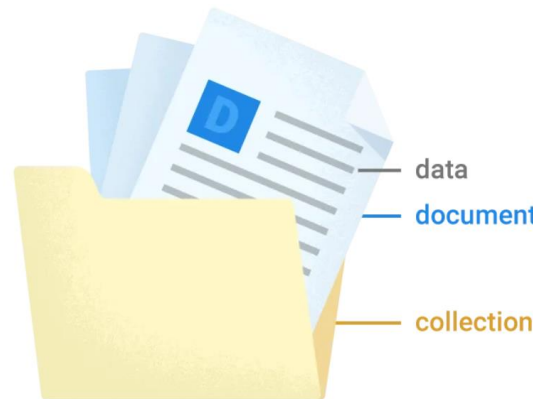
Firestore có kiến trúc theo mô hình phân cấp gồm bộ sưu tập (collections) và tài liệu (documents):

- **Collection (Bộ sưu tập):** Đây là tập hợp các tài liệu có cùng loại, giống như các bảng trong cơ sở dữ liệu quan hệ nhưng không cần một schema cố định. Mỗi bộ sưu tập chứa nhiều tài liệu, nhưng không chứa dữ liệu trực tiếp; chúng chỉ chứa các tài liệu. Cloud Firestore được tối ưu hoá để lưu trữ các bộ sưu tập lớn gồm nhiều tài liệu nhỏ.
- **Document (Tài liệu):** Đây là các đơn vị dữ liệu cơ bản nhất trong Firestore. Mỗi tài liệu được lưu dưới dạng khá giống JSON (tương tự như một đối tượng) và chứa các trường (fields) có thể là nhiều loại dữ liệu khác nhau: số, chuỗi, mảng, bản đồ, ... Mỗi tài liệu có một ID duy nhất và có thể chứa các bộ sưu tập con.

Các bộ sưu tập và tài liệu được tạo ngầm trong Cloud Firestore. Bạn chỉ cần chỉ định dữ liệu cho một tài liệu trong một tập hợp. Nếu bộ sưu tập hoặc tài liệu không tồn tại, Cloud Firestore sẽ tạo bộ sưu tập hoặc tài liệu đó.

Điểm nổi bật được nhiều lập trình viên yêu thích ở Firestore là hiệu suất cao, giao diện dễ sử dụng cũng như tùy chỉnh mở rộng linh hoạt. Có đa dạng ngôn ngữ được sử dụng khi lập trình trong Firestore, gồm có Java, C++, Unity, Go, Node.js SDKs, REST.

Firestore tổ chức dữ liệu theo cấu trúc phân cấp, cho phép tổ chức dữ liệu một cách linh hoạt. Collections -> Documents -> Subcollections -> Subdocuments,...



Hình 14: Kiến trúc của Firestore

Ví dụ: Dưới đây là 1 collection User

```
Users (Collection)
├─ userID_1 (Document)
│   ├── name: "John Doe"
│   ├── age: 30
│   └─ Orders (Subcollection)
│       ├── orderID_1 (Document)
│       │   ├── product: "Laptop"
│       │   └─ price: 1000
│       └─ ...
├─ userID_2 (Document)
└─ userID_3 (Document)
```

Hình 15: Ví dụ về 1 collection trong Firebase Firestore

2.2 Cơ chế hoạt động và một số tính năng của Firebase Firestore

Về cơ bản, Firestore hoạt động dựa trên mô hình NoSQL, thực hiện các truy vấn cho phép lưu trữ và truy xuất dữ liệu theo dạng tài liệu được sắp xếp logic thành các bộ sưu tập riêng biệt.

Firestore hoạt động dựa trên mô hình không yêu cầu cấu trúc bảng cố định như cơ sở dữ liệu quan hệ, thay vào đó sử dụng các tài liệu (documents) để lưu trữ thông tin, trong đó mỗi tài liệu là một cặp khóa-giá trị. Các tài liệu này lại được tập hợp thành các bộ sưu tập, cho phép lưu trữ các tài liệu có cấu trúc và loại dữ liệu khác nhau mà không cần thiết lập trước.

Dưới đây là một số tính năng của Firestore:

Truy vấn linh hoạt: Firestore cung cấp sự linh hoạt cao nhờ vào cấu trúc dữ liệu tài liệu, với các kiểu dữ liệu phong phú như chuỗi, số, mảng, đối tượng, boolean, và thậm chí cả trường địa lý (GeoPoint). Người dùng có thể tùy ý sử dụng và kết hợp các kiểu dữ liệu này trong một tài liệu để phù hợp với yêu cầu của ứng dụng. Firestore không hạn chế các tài liệu trong cùng một bộ sưu tập có cùng cấu trúc, điều này giúp dễ dàng thay đổi hoặc mở rộng dữ liệu mà không ảnh hưởng đến tài liệu khác. Một trong những điểm nổi bật của Firestore là khả năng thực hiện truy vấn linh hoạt và tối ưu hóa. Firestore cho phép người dùng lọc, sắp xếp và giới hạn kết quả tìm kiếm mà không làm giảm hiệu suất. Bên cạnh đó, Firestore có thể xử lý các truy vấn phức tạp như sử dụng điều kiện AND, OR, truy vấn theo nhiều trường và nhiều cấp độ trong tài liệu.

Đồng bộ dữ liệu thời gian thực: Firestore cung cấp các phương thức như get, set, update, delete để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên dữ liệu. Khi thực hiện một thao tác như cập nhật hoặc thêm mới, Firestore sẽ đồng bộ hóa thay đổi này ngay lập tức đến tất cả người dùng và thiết bị kết nối. Điều này có ý nghĩa quan trọng với các ứng dụng cần cập nhật dữ liệu liên tục, vì Firestore sẽ tự động đẩy dữ liệu mới lên tất cả các thiết bị, tránh việc lập trình viên phải gửi lại yêu cầu lên máy chủ để kiểm tra thay đổi.

Hơn thế nữa, chúng ta sẽ được gửi thông báo khi có người thay đổi dữ liệu hay cấu trúc trên hệ thống máy chủ. Chúng ta cũng sẽ không cần gửi yêu cầu lên hệ thống máy chủ mỗi khi cần thay đổi. Thay vào đó, Firestore sẽ tự động cập nhật đầy đủ tiến trình mà ta thực hiện.

Hỗ trợ làm việc ngoại tuyến: Firestore còn hỗ trợ tính năng đồng bộ hóa ngoại tuyến, giúp dữ liệu vẫn khả dụng và có thể cập nhật ngay cả khi thiết bị không có kết nối mạng. Khi kết nối được khôi phục, Firestore sẽ tự động đồng bộ hóa dữ liệu, đảm bảo rằng tất cả các thiết bị đều có dữ liệu mới nhất. Tính năng này đặc biệt hữu ích cho các ứng dụng di động, nơi mà người dùng có thể không có kết nối ổn định. Điều này giúp cải thiện trải nghiệm người dùng và làm cho Firestore trở thành lựa chọn phù hợp cho các ứng dụng cần hỗ trợ ngoại tuyến.

Tích hợp dễ dàng: Firestore tích hợp tốt với các dịch vụ khác của Firebase như Firebase Authentication, Firebase Cloud Functions và Firebase Hosting, giúp việc phát triển và triển khai ứng dụng trở nên thuận tiện và nhanh chóng.

Bảo mật và xác thực: Firestore cung cấp các cơ chế bảo mật và xác thực để kiểm soát quyền truy cập vào dữ liệu, bao gồm xác thực người dùng và quyền truy cập dựa trên vai trò.

Mở rộng và tin cậy: Firestore được xây dựng trên cơ sở hạ tầng đám mây của Google, cho phép mở rộng linh hoạt và đảm bảo tính tin cậy cao trong việc lưu trữ và quản lý dữ liệu.

So sánh Realtime Database và Firebase Firestore

Tiêu chí	Realtime Database	Firebase Firestore
Cấu trúc dữ liệu	Cấu trúc dạng cây JSON, đơn giản và phẳng	Dữ liệu được tổ chức theo tài liệu và bộ sưu tập, linh hoạt và phân cấp
Truy vấn dữ liệu	Chỉ hỗ trợ truy vấn cơ bản, giới hạn khả năng lọc và sắp xếp	Hỗ trợ truy vấn nâng cao với các khả năng phân trang, lọc và sắp xếp

Khả năng mở rộng	Khó mở rộng khi lượng dữ liệu tăng mạnh	Dễ dàng mở rộng, thích hợp cho ứng dụng phức tạp và quy mô lớn
Hỗ trợ ngoại tuyến	Hỗ trợ tốt cho ngoại tuyến ở mức cơ bản	Đồng bộ hóa dữ liệu khi ngoại tuyến tốt hơn
Chi phí	Phù hợp cho ứng dụng nhỏ, nhưng có thể tăng cao nếu mở rộng	Có thể cao hơn cho các truy vấn nâng cao
Sự dễ dàng sử dụng	Thích hợp cho ứng dụng đơn giản	Cần thời gian học hỏi nhưng cung cấp nhiều tính năng mạnh mẽ hơn

Bảng 3: So sánh Realtime Database và Firebase Firestore

2.3 Truy vấn và lọc dữ liệu trong Firestore

Firestore cung cấp một hệ thống truy vấn linh hoạt và mạnh mẽ:

- **Truy vấn (Querying):** Chúng ta có thể thực hiện các truy vấn để lọc dữ liệu bằng các điều kiện như where, orderBy, limit, startAt, và endAt.

Đây là một ví dụ về lọc dữ liệu user có điểm TOEIC ≥ 100 và sắp xếp theo tên.

Trong đó: orderBy: Sắp xếp kết quả theo tên theo thứ tự tăng dần.

whereGreaterThanOrEqualTo: Lọc các người dùng có điểm TOEIC từ 100 trở lên.

```

FirebaseFirestore db = FirebaseFirestore.getInstance();
db.collection( collectionPath: "users") CollectionReference
    .whereGreaterThanOrEqualTo( field: "toeic_score", value: 100) // Lọc điểm TOEIC >= 100
    .orderBy( field: "name") // Sắp xếp theo tên
    .get() Task<QuerySnapshot>
    .addOnSuccessListener(queryDocumentSnapshots -> {
        for (DocumentSnapshot document : queryDocumentSnapshots) {
            String name = document.getString( field: "name");
            int toeicScore = document.getLong( field: "toeic_score").intValue();
            // Xử lý dữ liệu (ví dụ, hiển thị tên và điểm TOEIC)
        }
    });

```

Hình 16: Ví dụ về lọc dữ liệu trong firebase


```

FirebaseFirestore db = FirebaseFirestore.getInstance();
db.collection( collectionPath: "users" ) CollectionReference
    .whereGreaterThanEqualTo( field: "toeic_score", value: 100) // Lọc điểm TOEIC >= 100
    .orderBy( field: "name" ) // Sắp xếp theo tên
    .limit(10) //lấy 10 người đầu tiên
    .get() Task<QuerySnapshot>
    .addOnSuccessListener(queryDocumentSnapshots -> {
        for (DocumentSnapshot document : queryDocumentSnapshots) {
            String name = document.getString( field: "name");
            int toeicScore = document.getLong( field: "toeic_score").intValue();
            // Xử lý dữ liệu (ví dụ, hiển thị tên và điểm TOEIC)
        }
    });

```

Hình 17: Ví dụ về lọc dữ liệu trong Firestore

Đây cũng tương tự như ví dụ trên nhưng khi thêm điều kiện limit(10) thì nó chỉ lấy 10 người đầu tiên trong danh sách user có điểm lớn hơn 100 được sắp xếp theo tên.

- **Kết hợp nhiều điều kiện:** Firestore cho phép kết hợp các điều kiện để tạo ra các truy vấn phức tạp. Ví dụ, chúng ta có thể lấy tất cả các user có điểm từ 100 trở lên và sắp xếp theo tên như ví dụ trên.
- **Chỉ mục (Indexes):** Các điều kiện trong truy vấn yêu cầu phải tạo chỉ mục để đảm bảo hiệu năng. Firestore tự động tạo chỉ mục cơ bản, và bạn có thể tạo chỉ mục tùy chỉnh nếu cần.

2.4 Quy tắc bảo mật trong Firebase Firestore

Firestore dùng quy tắc bảo mật để kiểm soát quyền truy cập vào dữ liệu. Bạn có thể thiết lập quy tắc để chỉ người dùng đã xác thực mới có thể đọc/ghi vào dữ liệu của họ hoặc áp dụng các điều kiện đặc biệt để kiểm soát chặt chẽ hơn.

Quy tắc bảo mật trong Firestore được xác định thông qua các điều kiện trong các tệp firestore.rules trong Firebase Console. Quy tắc này cho phép bạn chỉ định quyền truy cập đọc/ghi dựa trên các điều kiện cụ thể.

Ví dụ về quy tắc bảo mật cơ bản:

```

service cloud.firestore {
  match /databases/{database}/documents {

    // Quy tắc cho phép đọc/ghi tài liệu trong collection 'users' chỉ khi người dùng đã xác
    match /users/{userId} {
      // Người dùng phải xác thực và chỉ có thể truy cập tài liệu của chính họ
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Quy tắc cho phép bất kỳ ai đọc tài liệu trong collection 'public'
    match /public/{document=**} {
      allow read: if true; // Chỉ đọc công khai
      allow write: if false; // Không ai có thể ghi vào collection 'public'
    }
  }
}

```

Hình 18: Ví dụ về quy tắc bảo mật cơ bản trong firestore

request.auth != null: Điều kiện này đảm bảo rằng người dùng phải xác thực (đã đăng nhập) mới có quyền truy cập vào dữ liệu.

request.auth.uid == userId: Điều kiện này đảm bảo rằng người dùng chỉ có thể đọc hoặc ghi vào tài liệu của chính họ. Ví dụ, chỉ người dùng có uid tương ứng với tài liệu có ID userId mới có thể thay đổi hoặc đọc tài liệu đó.

match /public/{document=}:** Điều này cho phép tất cả người dùng có thể đọc tài liệu trong collection public, nhưng không ai có thể ghi vào đó.

2.5 Ưu và nhược điểm của Firebase Firestore

Bên cạnh những ưu điểm vượt trội, Firestore còn tồn tại một số hạn chế mà doanh nghiệp cần lưu ý để sử dụng hiệu quả hơn.

Ưu điểm	Hạn chế
Có khả năng lưu trữ nhiều loại dữ liệu như tài liệu và tập hợp mà không yêu cầu cấu trúc dữ liệu cố định.	Không hỗ trợ các truy vấn phức tạp như JOIN, subquery và aggregation như một số hệ quản trị cơ sở dữ liệu truyền thống.

Cập nhật chi tiết dữ liệu dựa theo thời gian thực .	Nếu không kiểm soát dữ liệu lưu trữ và số lượng đọc/ghi thì chi phí sử dụng có thể vượt mức và tăng nhanh.
Linh hoạt mở rộng dễ dàng theo quy mô và nhu cầu của doanh nghiệp.	Có giới hạn về số lượng giao dịch có thể xử lý.
Hỗ trợ khả năng đồng bộ dữ liệu khi không có kết nối mạng, giúp nâng cao trải nghiệm người dùng.	Không có tính năng sao lưu tự động, cần tích hợp với ứng dụng của bên thứ ba để sao lưu dữ liệu.
Cung cấp các tính năng bảo mật vượt trội, mạnh mẽ.	Dữ liệu có thể bị phân mảnh, gây khó khăn trong việc duy trì tính toàn vẹn.
Giao diện dễ sử dụng và thân thiện với người dùng.	Phụ thuộc nhiều vào hệ sinh thái Google Cloud, làm giảm tính linh hoạt khi tích hợp với các nền tảng khác.
Tích hợp với các ứng dụng dịch vụ của Google Cloud Platform giúp tối ưu năng suất và tận dụng tối đa các tính năng.	Người mới tiếp cận mô hình NoSQL và Firestore có thể cần thời gian làm quen với cấu trúc và cách hoạt động của hệ thống.
Có nhiều tài liệu và cộng đồng hỗ trợ phong phú.	

Bảng 4: Ưu và nhược điểm của Firestore

CHƯƠNG 3 – TỔNG QUAN VỀ DỰ ÁN

3.1 Giới thiệu về dự án

Dự án này nhằm mục đích có được sự hiểu biết thấu đáo về Firebase Firestore để phát triển một ứng dụng cung cấp các chức năng khác nhau liên quan đến quản lý thông tin sinh viên. Chức năng phần mềm chính chia làm 3 phần:

- **User Account Management (Quản lý tài khoản người dùng):** bao gồm các chức năng đăng nhập, cập nhật ảnh người dùng, xem danh sách người dùng, thêm người dùng mới, xóa người dùng, chỉnh sửa thông tin và xem lịch sử đăng nhập của user.
- **Student Management (Quản lý sinh viên):** bao gồm các chức năng xem danh sách sinh viên, thêm sinh viên mới, xóa sinh viên, sửa thông tin sinh viên, sắp xếp danh sách sinh viên dựa trên nhiều tiêu chí, tìm kiếm sinh viên bằng nhiều tiêu chí, truy cập trang cá nhân của sinh viên, xem thông tin sinh viên trên một màn hình chi tiết, quản lý danh sách chứng chỉ của sinh viên, xem danh sách chứng chỉ, xem thông tin chứng chỉ, sửa thông tin chứng chỉ, xóa chứng chỉ.
- **Data Import/Export (Nhập/ xuất dữ liệu):** Import danh sách sinh viên từ một tệp, xuất danh sách sinh viên sang Excel/CSV, nhập danh sách chứng chỉ cho học sinh từ tệp, xuất danh sách chứng chỉ sang Excel/CSV.

3.2 Đối tượng người dùng của dự án

Ứng dụng cung cấp ba vai trò người dùng: quản trị viên, người quản lý và nhân viên. Tài khoản admin được tích hợp với hệ thống mà không cần tạo. Các tài khoản còn lại có thể được tạo bởi quản trị viên, người dùng bình thường không thể tự tạo tài khoản. Trong các chức năng nêu trên, admin có thể thực hiện mọi thứ, người quản lý có thể thực hiện tất cả các chức năng liên quan đến sinh viên, và tài khoản nhân viên chỉ có thể xem nội dung. Nhân viên không được phép chỉnh sửa bất kỳ nội dung nào khác ngoài việc cập nhật ảnh hồ sơ của họ.

3.3 Công nghệ sử dụng trong dự án và lý do sử dụng

➤ **Firestore:** Lý do chọn Firestore làm cơ sở dữ liệu vì:

- **Tính mở rộng (Scalability):** Firestore là cơ sở dữ liệu NoSQL có khả năng mở rộng linh hoạt, phù hợp với các ứng dụng cần xử lý một lượng dữ liệu lớn và tăng trưởng nhanh chóng. Firestore tự động quản lý việc mở rộng mà không cần phải lo lắng về việc duy trì phần cứng hoặc cấu hình cơ sở dữ liệu phức tạp. Firestore có thể xử lý khối lượng dữ liệu khổng lồ mà vẫn đảm bảo hiệu suất ổn định, từ đó phù hợp cho các ứng dụng có lượng người dùng lớn và yêu cầu khả năng mở rộng cao.
- **Khả năng đồng bộ dữ liệu thời gian thực:** Một trong những điểm mạnh của Firestore là khả năng đồng bộ dữ liệu theo thời gian thực. Điều này có nghĩa là khi có thay đổi trong cơ sở dữ liệu (thêm, sửa, xóa dữ liệu), tất cả các client đã kết nối sẽ nhận được thông báo ngay lập tức và cập nhật dữ liệu mà không cần phải làm mới lại ứng dụng. Điều này rất hữu ích trong các ứng dụng cần hiển thị dữ liệu cập nhật liên tục, ví dụ như quản lý sinh viên, nơi người dùng muốn có cái nhìn chính xác và kịp thời về trạng thái dữ liệu.
- **Tính bảo mật cao:** Firestore cung cấp hệ thống bảo mật mạnh mẽ thông qua quy tắc bảo mật (Security Rules). Bạn có thể thiết lập quyền truy cập rất chi tiết và linh hoạt, chẳng hạn như chỉ cho phép người dùng đã đăng nhập có quyền truy cập vào dữ liệu của mình. Quy tắc bảo mật có thể áp dụng dựa trên thông tin xác thực người dùng, ví dụ: `request.auth.uid` sẽ đảm bảo rằng người dùng chỉ có thể truy cập hoặc thay đổi dữ liệu của chính họ. Firestore còn tích hợp với Firebase Authentication để kiểm soát quyền truy cập và bảo vệ dữ liệu, giúp xây dựng các ứng dụng an toàn và bảo mật hơn.

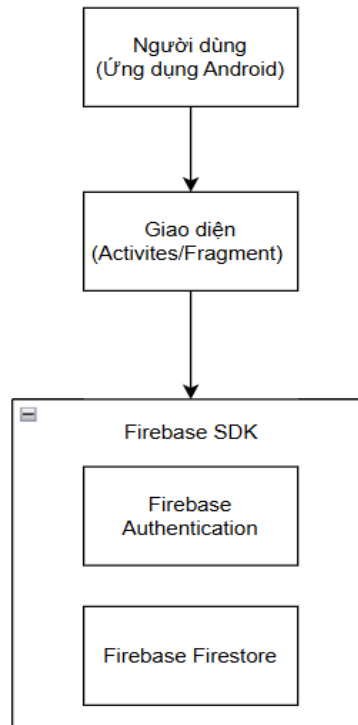
➤ **Authentication:**

- **Xác thực người dùng:** Authentication cung cấp nhiều phương thức xác thực như email/password, Google Sign-In, Facebook Login, và các phương thức OAuth khác, giúp việc quản lý người dùng trở nên đơn giản và bảo mật. Dự án của

chúng em sẽ sử dụng Email/Password Authentication để người dùng có thể đăng nhập vào hệ thống bằng tài khoản của mình. Người dùng chỉ cần nhập email và mật khẩu, Firebase sẽ chịu trách nhiệm xác thực và quản lý phiên làm việc.

CHƯƠNG 4 – KIẾN TRÚC VÀ TRIỂN KHAI DỰ ÁN

4.1 Kiến trúc tổng thể của hệ thống



Hình 19: Kiến trúc tổng thể của hệ thống

- **Giao diện (Activities/Fragments):**

- Activities và Fragments là nơi người dùng tương tác với ứng dụng, chẳng hạn như màn hình đăng nhập, danh sách sinh viên, thêm, sửa, xóa sinh viên.
- Các giao diện này kết nối trực tiếp với Firestore thông qua Firebase SDK mà không cần lớp backend như trong web. Điều này giúp tối giản cấu trúc hệ thống và tận dụng các API của Firebase SDK để dễ dàng thực hiện các truy vấn trực tiếp từ ứng dụng.

- **Firebase SDK:**

- Firebase SDK cung cấp tất cả các thư viện và công cụ cần thiết để Android có thể kết nối với Firebase, thực hiện các thao tác xác thực và lưu trữ dữ liệu.

- **Firestore Authentication:** Firestore Authentication xử lý toàn bộ phần xác thực người dùng trong ứng dụng Android. Người dùng có thể đăng nhập qua nhiều phương thức (email/password, Google, Facebook, v.v.), và bạn có thể dễ dàng quản lý quyền truy cập dựa trên UID của người dùng. Sau khi đăng nhập, Firestore Authentication sẽ cung cấp mã ID Token để xác minh người dùng. Bạn có thể kiểm tra mã này để cho phép truy cập vào các tài nguyên nhất định trong Firestore.
- **Firestore Database:** Firestore là cơ sở dữ liệu NoSQL chính của ứng dụng, nơi lưu trữ tất cả thông tin sinh viên, thông tin tài khoản người dùng, và các dữ liệu khác liên quan. Android SDK hỗ trợ truy vấn trực tiếp đến Firestore để lấy dữ liệu (ví dụ: lấy danh sách sinh viên, truy xuất thông tin chi tiết sinh viên) mà không cần backend trung gian. Điều này giúp quá trình đồng bộ dữ liệu diễn ra nhanh chóng và trực tiếp trên ứng dụng.

4.2 Thiết kế cơ sở dữ liệu

- **Collection accounts:** Đây là collection để lưu trữ tài khoản của người dùng. Mỗi document trong collection accounts đại diện cho một tài khoản của user, với một ID duy nhất.

Các trường trong document của accounts:

email (String): Email user (ví dụ: "nguyenhamyjenna2007@gmail.com")

name (String): Tên user (ví dụ: "Nguyen Van A")

role (String): Vai trò của user (ví dụ: "admin" hoặc "manager" hoặc "employee")

- **Collection loginHistory:** Đây là collection để lưu trữ thông tin lịch sử đăng nhập của người dùng. Mỗi document trong collection loginHistory đại diện cho một tài khoản khi đăng nhập vào ứng dụng, với một ID duy nhất.

Các trường trong document của loginHistory:

email (String): Email user (ví dụ: "nguyenhamyjenna2007@gmail.com")

username (String): Tên user (ví dụ: "Nguyen Van A")

loginTime (String): Thời gian user đăng nhập vào ứng dụng (ví dụ: "2024-11-15 16:06:15")

avatar (String): Dữ liệu hình ảnh của sinh viên được lưu dưới dạng chuỗi Base64 (để hiển thị ảnh đại diện)

- **Collection students:** Đây là collection để lưu trữ thông tin của từng sinh viên. Mỗi document trong collection students đại diện cho một sinh viên, với một ID duy nhất. Các trường trong document của students:

id (String): Mã sinh viên (ví dụ: "52200001")

name (String): Tên sinh viên (ví dụ: "Nguyen Van A")

class (String): Lớp của sinh viên (ví dụ: "22050201")

faculty (String): Khoa của sinh viên (ví dụ: "CNTT")

birthday (String): Ngày sinh của sinh viên (ví dụ: "2/2/2004")

email (String): Địa chỉ email của sinh viên (ví dụ: "a@gmail.com")

phone (String): Số điện thoại của sinh viên (ví dụ: "0123456789")

profileImageBase64 (String): Dữ liệu hình ảnh của sinh viên được lưu dưới dạng chuỗi Base64 (để hiển thị ảnh đại diện)

- **Collection users:** Collection users lưu trữ thông tin của người dùng (bao gồm quản trị viên, người quản lý và nhân viên) để xác thực và phân quyền. Mỗi document trong users đại diện cho một người dùng với ID duy nhất (UID được tạo ra bởi Firebase Authentication khi người dùng đăng ký).

Các trường trong document của users:

name (String): Tên người dùng.

email (String): Địa chỉ email của người dùng.

role (String): Vai trò của người dùng (ví dụ: "admin" hoặc "manager" hoặc "employee"), giúp xác định quyền truy cập.

status (String): normal/locked.

birthday (String): Ngày sinh của người dùng (ví dụ: "2/2/2004").

phone (String): Số điện thoại của người dùng (ví dụ: "0123456789").

profileImageBase64 (String): Dữ liệu hình ảnh của sinh viên được lưu dưới dạng chuỗi Base64 (để hiển thị ảnh đại diện).

- **Collection certificates:** Collection certificates lưu trữ thông tin certificate của sinh viên.

Các trường trong document của certificates:

cername (String): Tên chứng chỉ.

des (String): Mô tả nội dung chứng chỉ.

organization (String): Tổ chức cấp chứng chỉ.

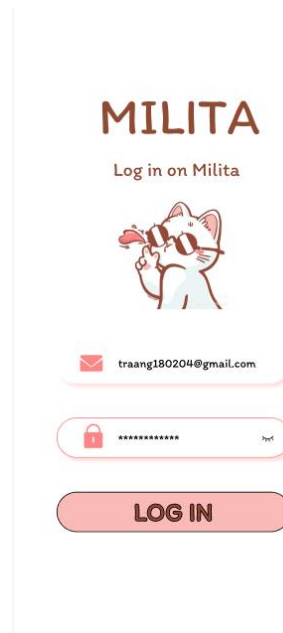
school (String): Trường mà sinh viên đang theo học.

session (String): Ngày tạo chứng chỉ.

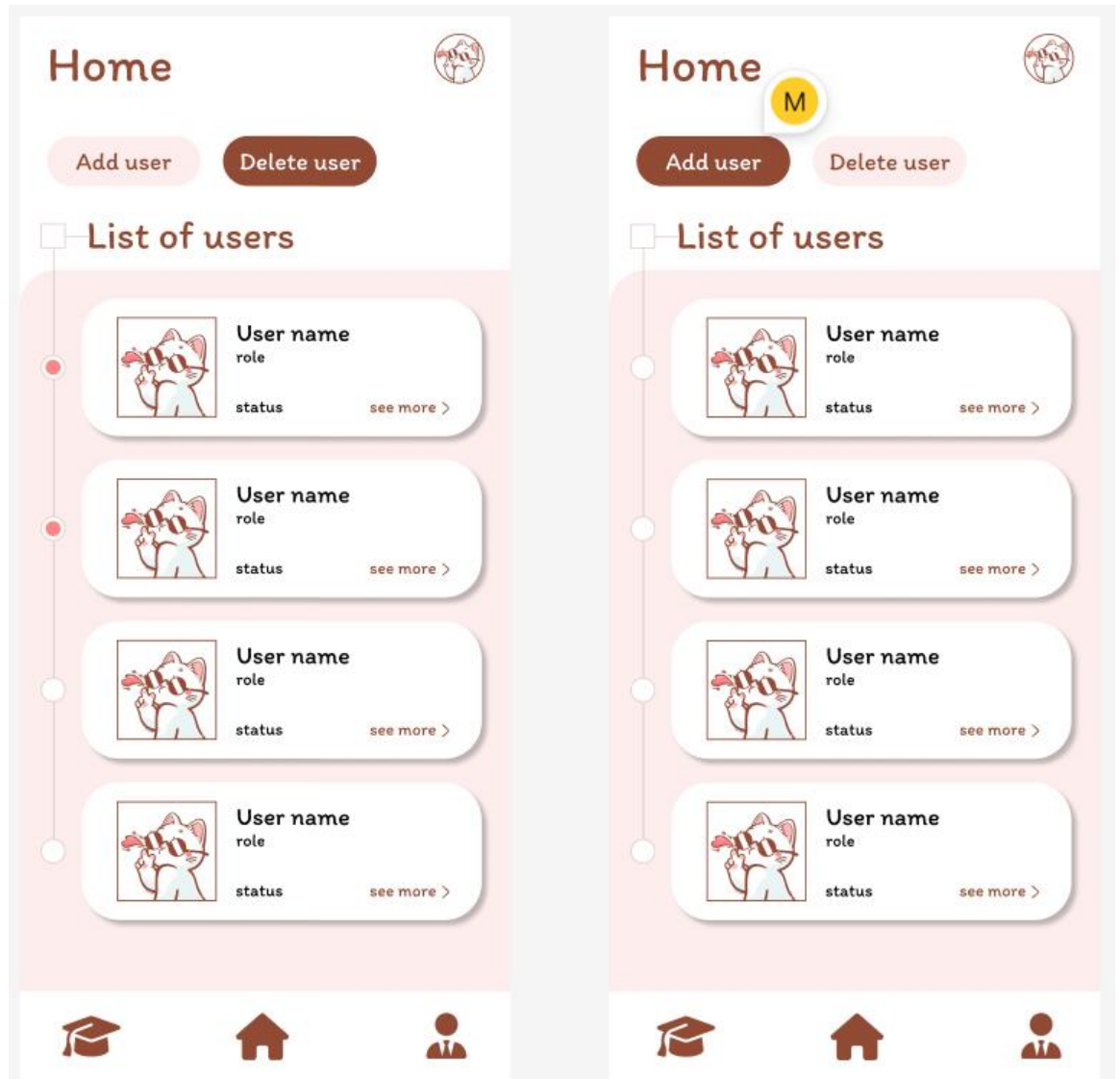
profileImageBase64 (String): Dữ liệu hình ảnh chứng chỉ của sinh viên được lưu dưới dạng chuỗi Base64 (để hiển thị ảnh)

4.3 Thiết kế giao diện

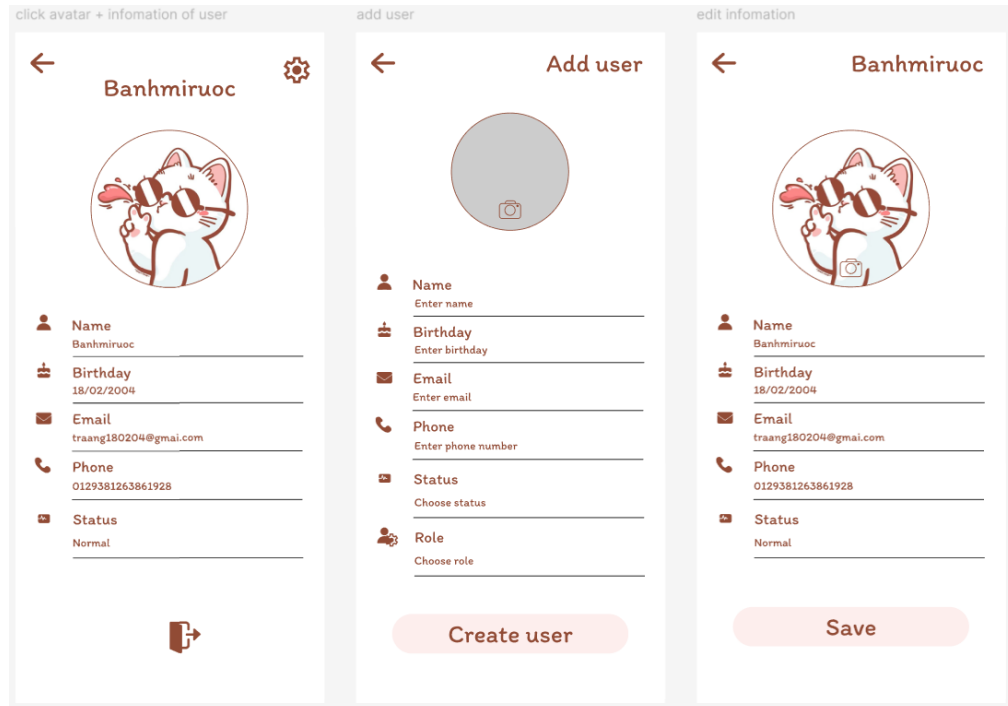
- Quản lý tài khoản người dùng



Hình 20: Màn hình login

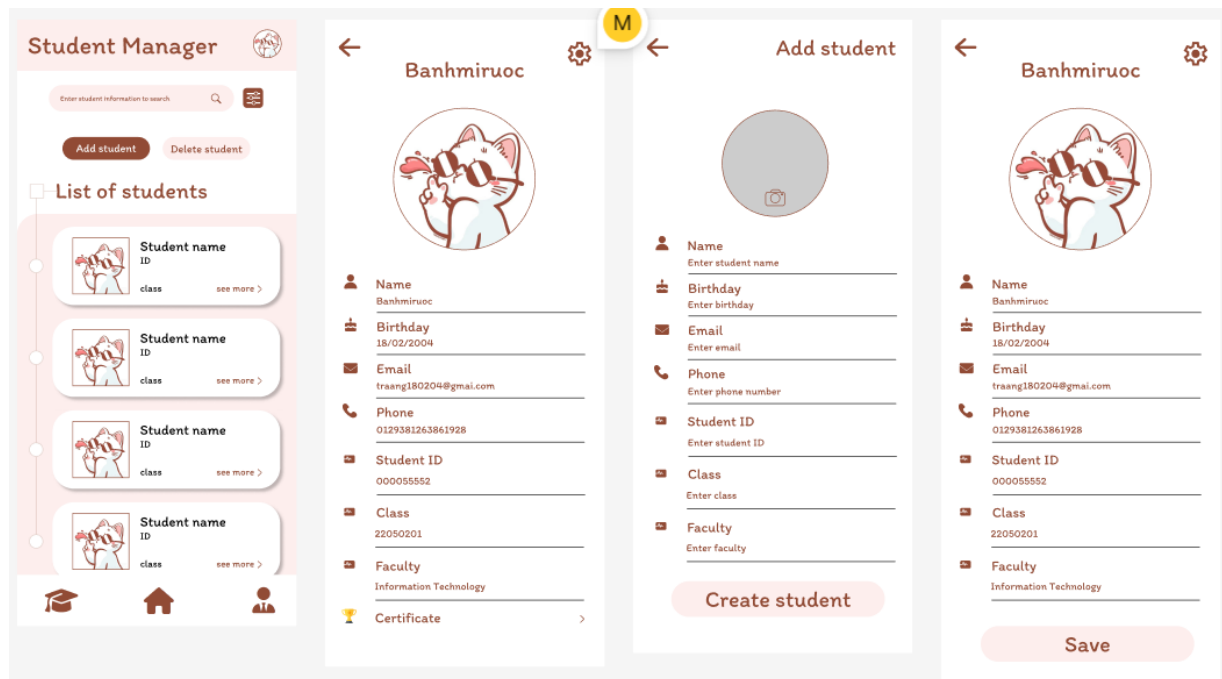


Hình 21: Màn hình trang chủ quản lý tài khoản người dùng

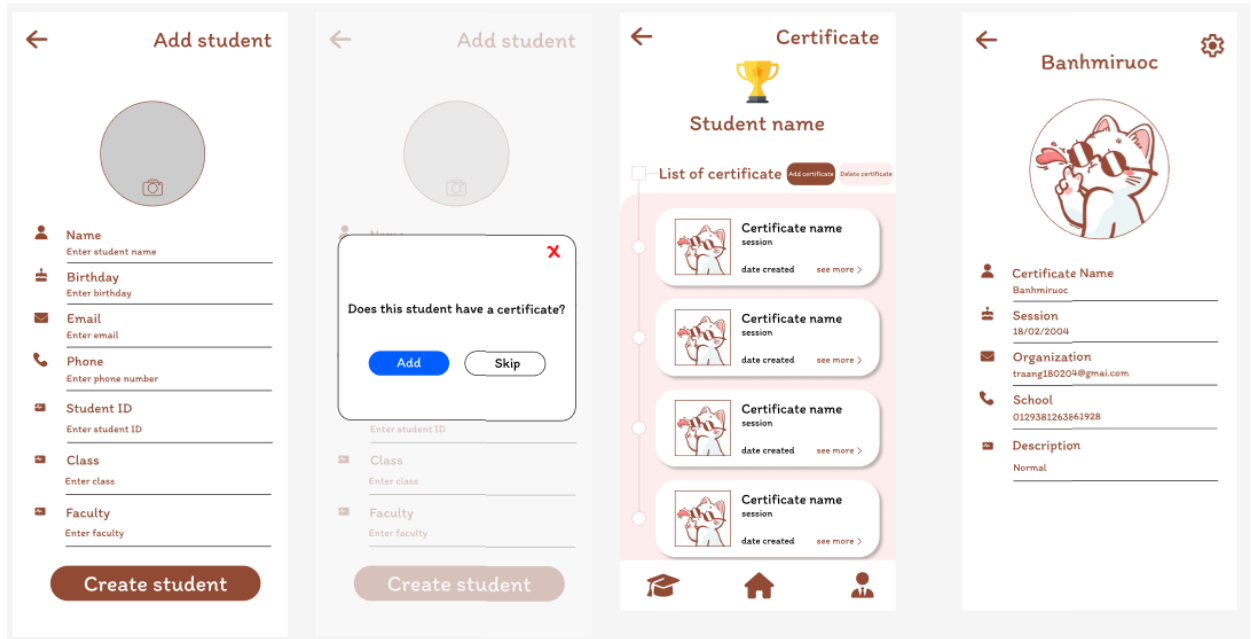


Hình 22: Màn hình xem chi tiết, thêm mới và sửa thông tin người dùng

- Quản lý sinh viên



Hình 23: Màn quản lý, xem chi tiết, thêm mới và sửa thông tin sinh viên

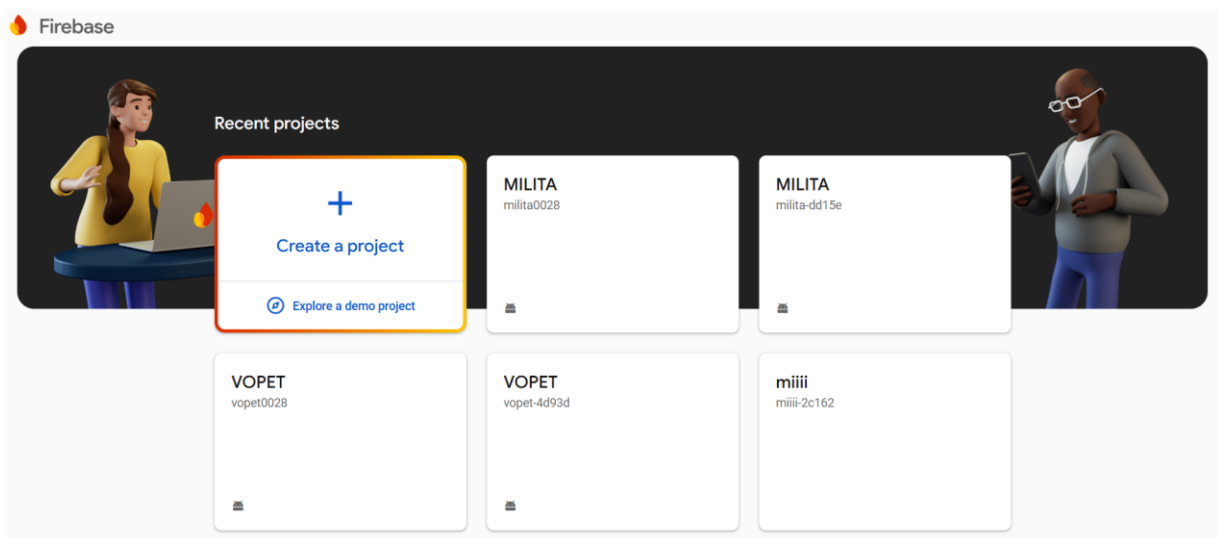


Hình 24: Màn hình quản lý, xem chi tiết, thêm mới và sửa thông tin chứng chỉ

4.4 Quy trình triển khai

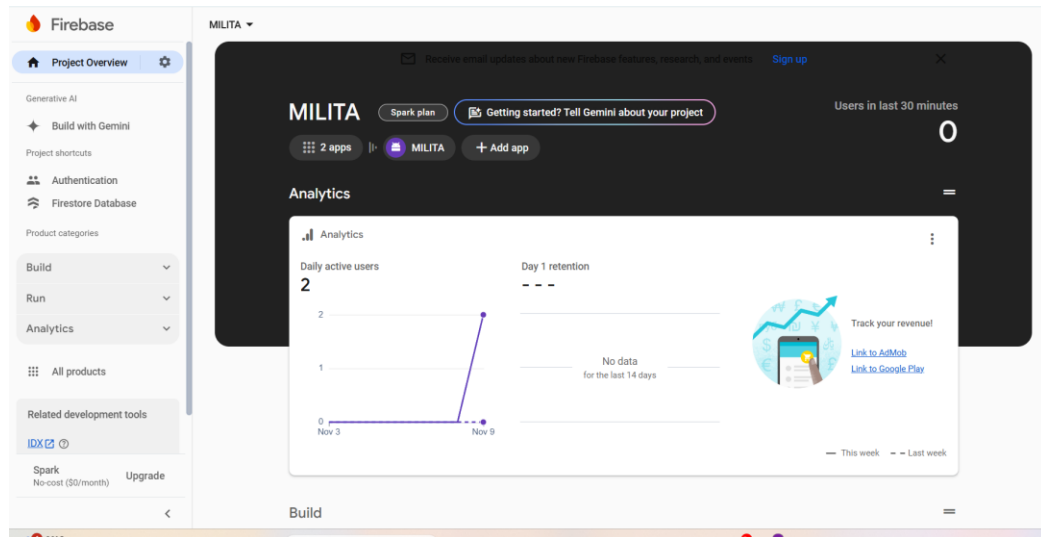
4.4.1 Tích hợp firebase vào ứng dụng

Để tạo một Firebase project mới, truy cập vào Firebase Console, đăng nhập với tài khoản Google của mình, và nhấn CREATE NEW PROJECT và tạo project.



Hình 25: Quy trình tích hợp firebase vào ứng dụng

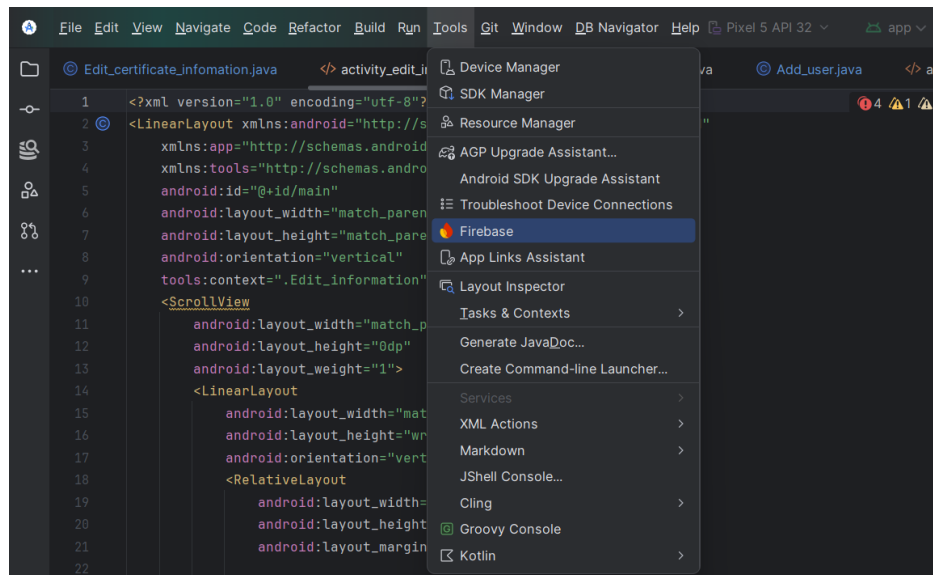
Sau khi tạo xong, sẽ thấy giao diện như sau:



Hình 26: Quy trình tích hợp firebase vào ứng dụng

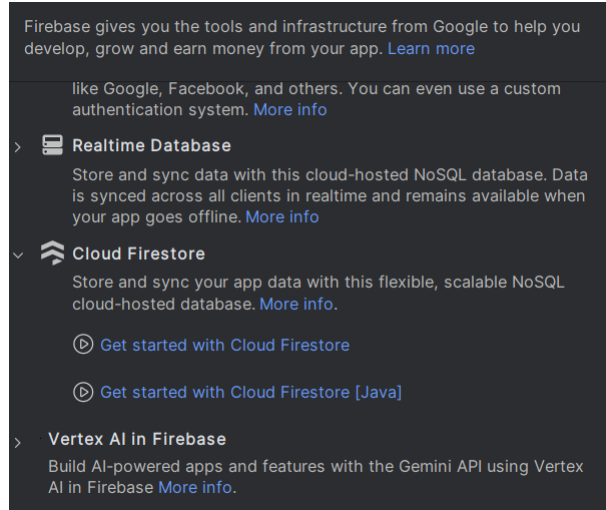
Sẽ có 2 bước kết nối project của mình với firebase đó là kết nối nhờ tools trong android studio hoặc kết nối ngay trên firebase, nhưng ở đây chúng em sẽ kết nối nhờ tools trong android studio.

Để kết nối với project với firebase, chúng ta sẽ mở project của mình ra mà bấm chọn vào tools trên thanh công cụ, và bấm chọn firestore như ảnh dưới đây:



Hình 27: Quy trình tích hợp firebase vào ứng dụng

Sau khi bấm chọn firestore thì màn hình này sẽ hiện ra và chúng ta sẽ tìm mục có tên là cloud firestore:



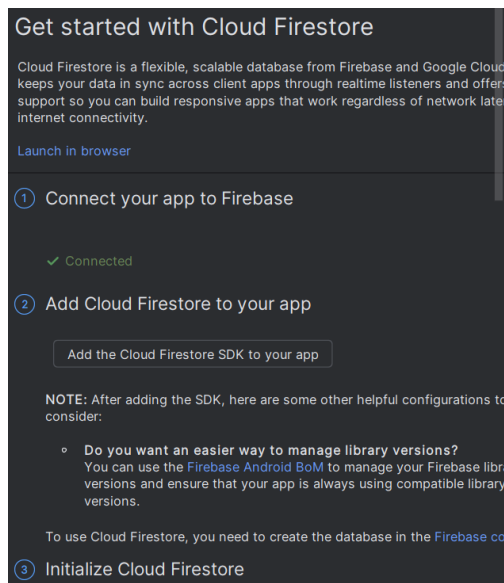
Hình 28: Quy trình tích hợp firebase vào ứng dụng

Sau đó ta sẽ kết nối với firestore theo các bước sau:

Bước 1: Connect your app to firebase

Bước 2: Add the Cloud Firestore SDK to our app

Sau khi hiện connected thì thành công.

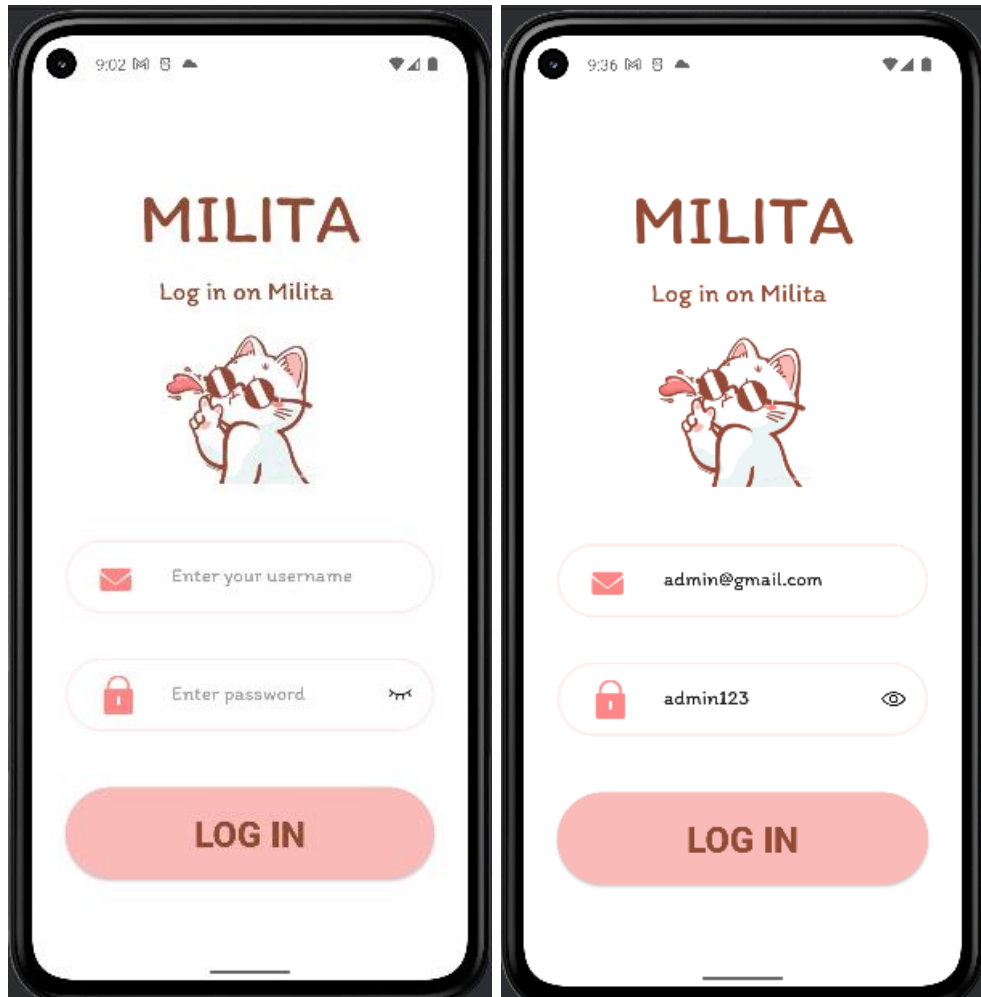


Hình 29: Quy trình tích hợp firebase vào ứng dụng

Sau đó chúng ta sẽ lên firebase kiểm tra lại project của mình đã có trong firebase hay chưa.

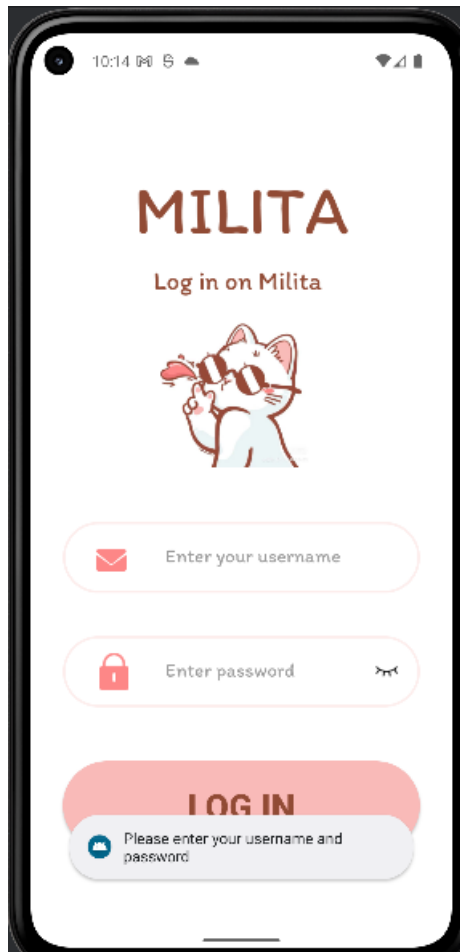
4.4.2 Hiện thực ứng dụng

- Chức năng quản lý tài khoản người dùng



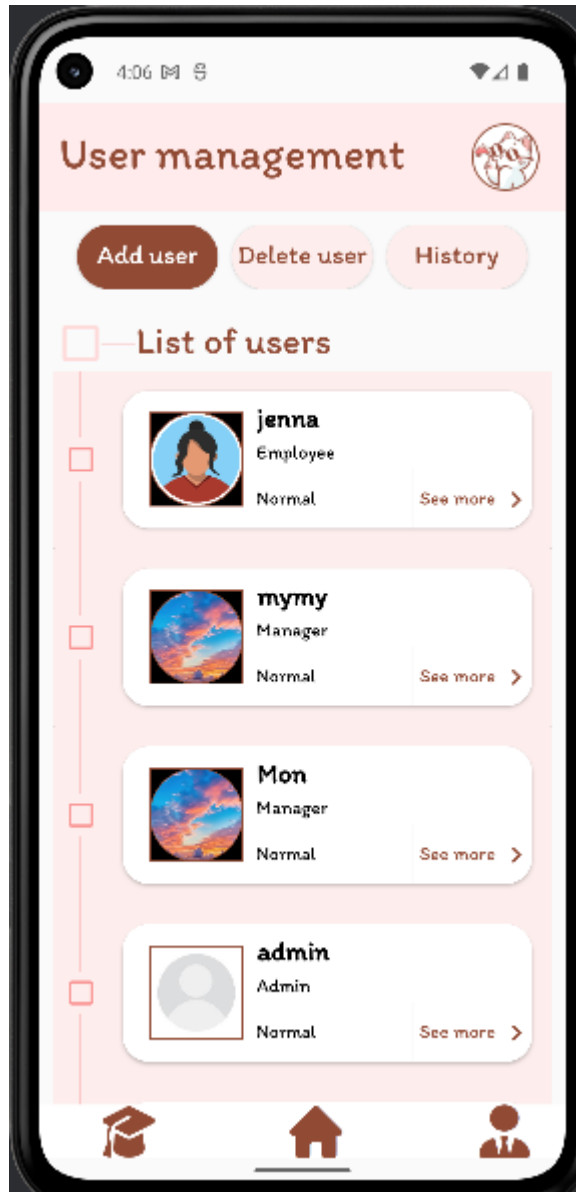
Hình 30: Màn hình Login

Khi người dùng không nhập username và password sẽ thông báo cho người dùng biết.



Hình 31: Validate màn hình Login

Khi đăng nhập thành công sẽ xuất hiện màn hình sau đối với tài khoản admin, đối với tài khoản admin sẽ có quyền liên quan đến quản lý sinh viên và quản lý người dùng. Còn khi đăng nhập với tài khoản manager thì chỉ có quyền liên quan đến quản lý sinh viên. Khi đăng nhập với tài khoản employee thì chỉ có quyền thay đổi ảnh đại diện của employee đó.



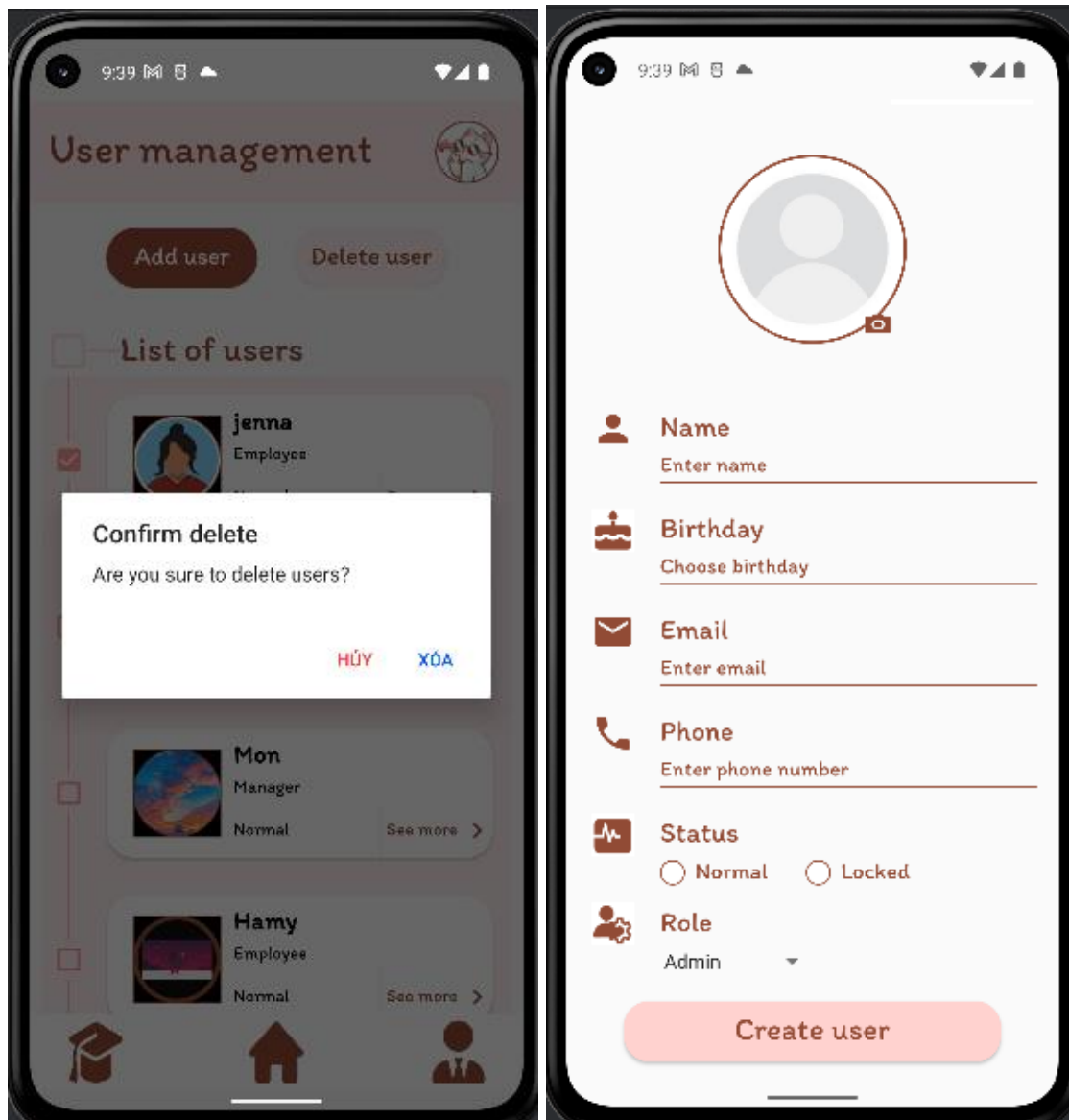
Hình 32: Màn hình quản lý tài khoản người dùng

Khi bấm vào nút see more trên mỗi user thì sẽ hiện ra thông tin chi tiết của user đó như hình bên dưới và khi bấm vào icon bánh răng trên góc phải ở trên thì sẽ xuất hiện trang chỉnh sửa thông tin người dùng, sau khi chỉnh sửa thì bấm nút Save để lưu thông tin đã chỉnh sửa được lưu vào firestore.



Hình 33: Màn hình xem và sửa thông tin tài khoản người dùng

Khi bấm chọn vào checkbox của mỗi user và bấm nút delete user sẽ hiện dialog để confirm lại việc xóa user của admin như hình bên dưới, admin có quyền xóa hoặc không. Nếu admin muốn xóa hết tất cả danh sách user thì sẽ bấm vào checkbox trên cùng để thực hiện xóa hết. Khi bấm vào nút add user thì trang add user sẽ hiện ra, sau đó admin sẽ thực hiện thêm các thông tin của user và bấm nút create user thì user sẽ được tạo và lưu vào firestore.



Hình 34: Màn hình thêm và xóa người dùng

- Chức năng quản lý sinh viên



Hình 35: Màn hình quản lý student và chức năng tìm kiếm



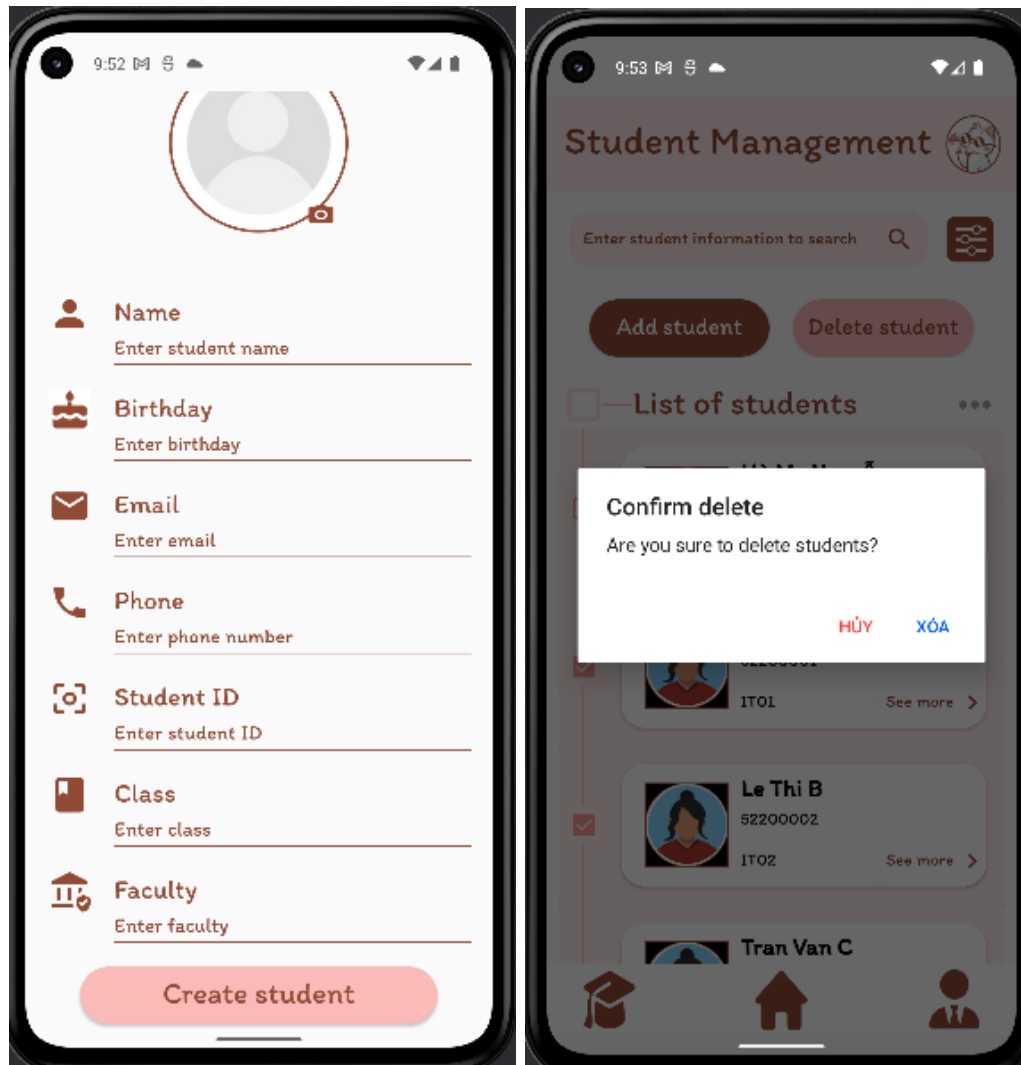
Hình 36: Chức năng lọc sinh viên theo nhiều tiêu chí

Hình bên trái là chức năng lọc sinh viên, có theo tiêu chí lọc sinh viên như sau: lọc theo tên từ A-Z và từ Z-A, lọc theo ID của sinh viên là tăng dần hoặc giảm dần. Hình bên phải là lọc tên sinh viên từ A-Z. Tên ở đây là tính cả họ và tên.



Hình 37: Chức năng lọc sinh viên theo tên và ID

Hình bên trái là chức năng lọc sinh viên theo tên từ Z-A. Hình bên phải là lọc theo ID tăng dần.



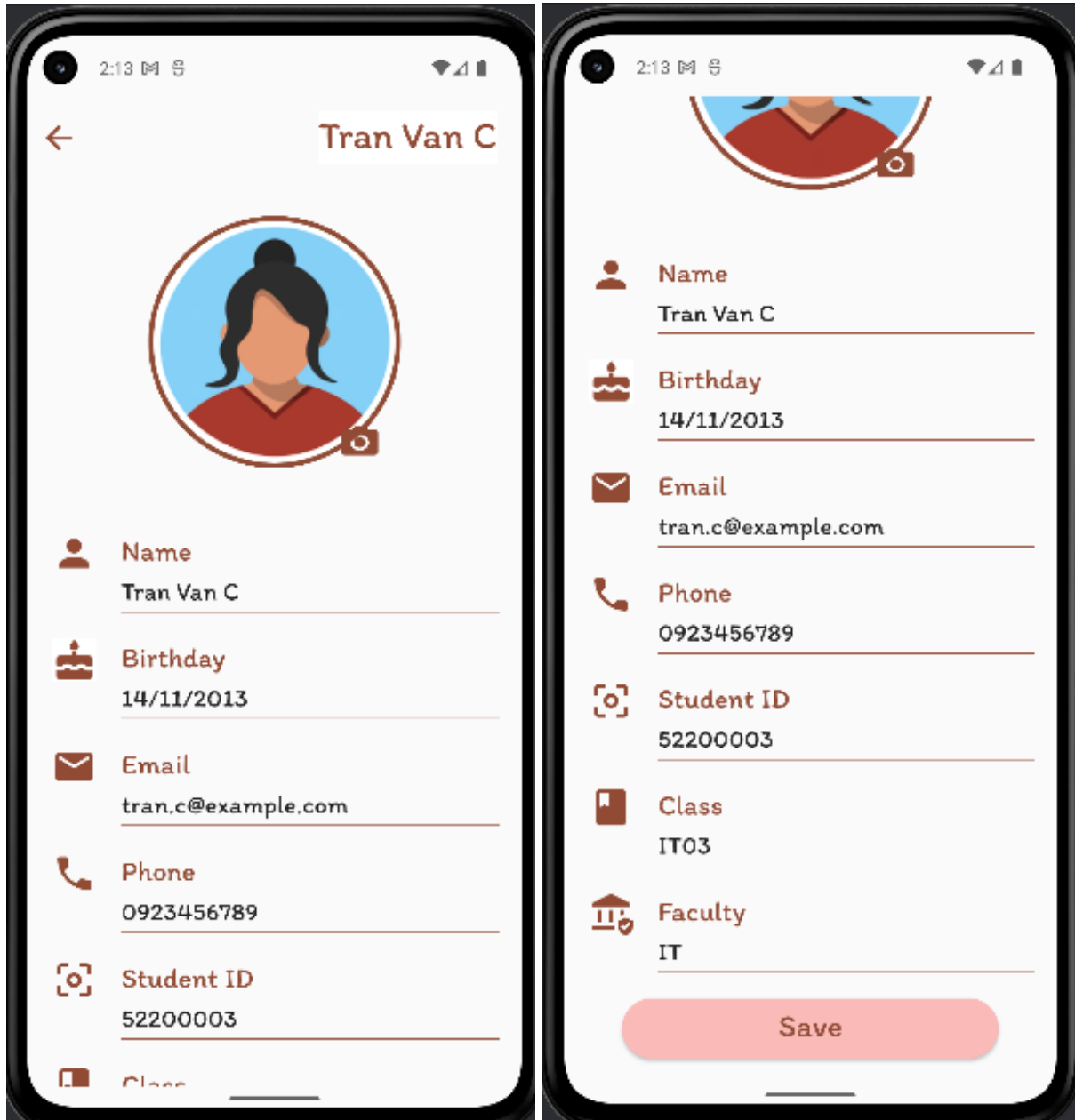
Hình 38: Màn hình thêm student và xóa student

Tương tự như quản lý user, khi bấm chọn vào checkbox của mỗi student và bấm nút delete student sẽ hiện dialog để confirm lại việc xóa student của admin như hình bên dưới, admin có quyền xóa hoặc không. Nếu admin muốn xóa hết tất cả danh sách student thì sẽ bấm vào checkbox trên cùng để thực hiện xóa hết. Khi bấm vào nút add student thì trang add student sẽ hiện ra, sau đó admin sẽ thực hiện thêm các thông tin của student và bấm nút create student thì student sẽ được tạo và lưu vào firestore.



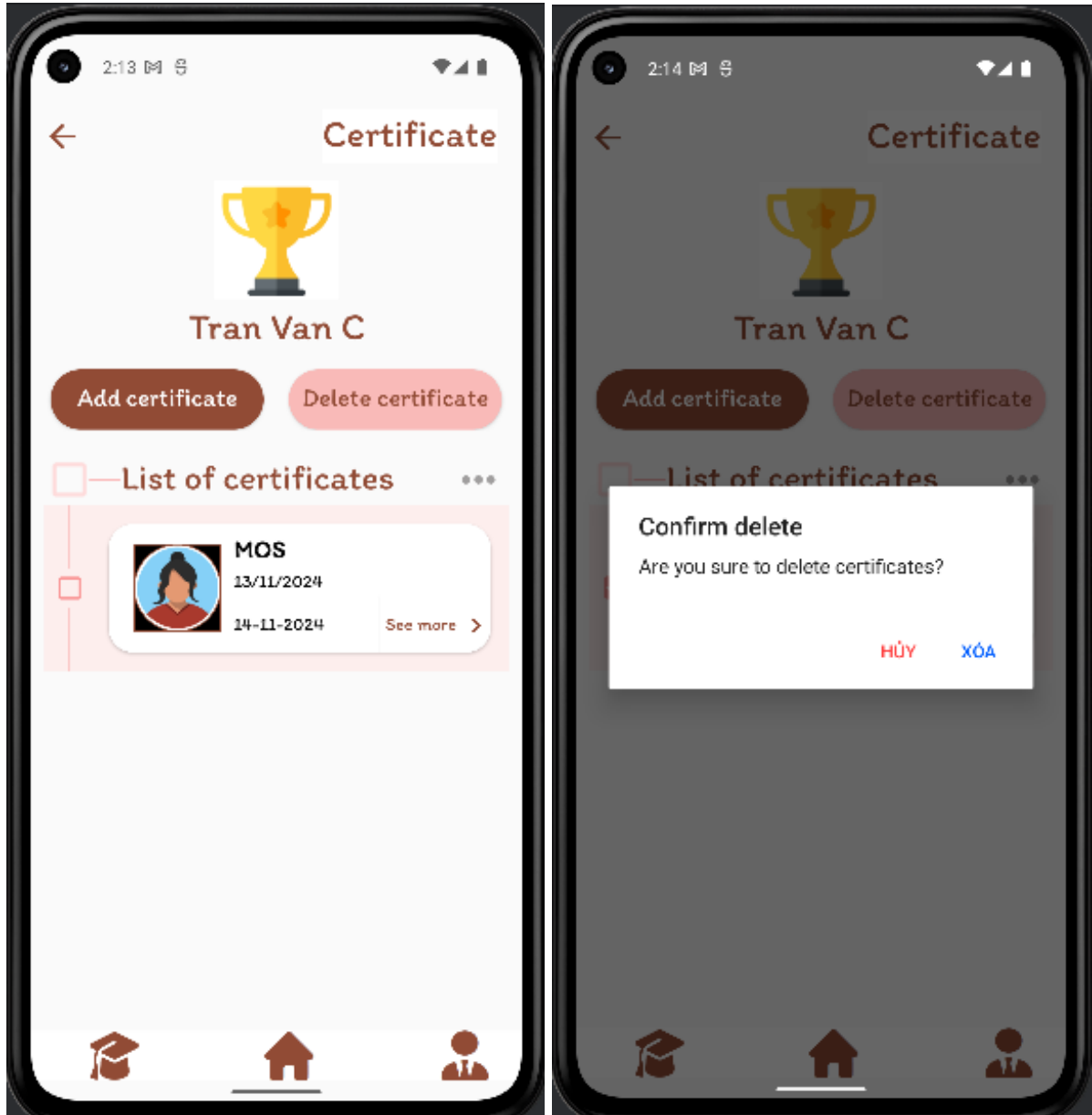
Hình 39: Màn hình xem thông tin chi tiết của student

Tương tự như quản lý người dùng, khi bấm vào nút see more sẽ hiện ra trang thông tin chi tiết của sinh viên.



Hình 40: Màn hình chỉnh sửa thông tin chi tiết của student

Tương tự như quản lý người dùng, khi bấm vào icon bánh răng ở góc trên bên phải của màn hình xem chi tiết thì sẽ hiện ra trang chỉnh sửa thông tin chi tiết của sinh viên. Sau khi sửa thông tin và bấm nút Save thì thông tin sẽ được cập nhật và firestore.



Hình 41: Màn hình xem thông tin chứng chỉ của student

Khi bấm vào certificate ở trang xem thông tin chi tiết của sinh viên thì sẽ hiện giao diện như hình bên trái, chứa list certificate của sinh viên đó. Muốn xóa certificate thì bấm chọn vào checkbox của certificate đó và bấm nút delete thì sẽ hiện dialog confirm như hình bên phải để xác nhận việc xóa của admin hoặc manager.

The image displays two mobile application screens for adding a certificate. Both screens feature a white background with a light gray header bar.

Left Screen: The header bar contains a back arrow on the left and the title "Add certificate" in the center. Below the header is a circular profile picture placeholder. The main content area lists six fields, each with an icon and a label:

- Certificate Name:** Icon of a trophy, label "Certificate Name", input field "Enter certificate name".
- Student Name:** Icon of a person, label "Student Name", input field "Tran Van C".
- Student ID:** Icon of a document with a magnifying glass, label "Student ID", input field "52200003".
- Session:** Icon of a calendar, label "Session", input field "Enter session".
- Organization:** Icon of an envelope, label "Organization", input field "Enter organization".
- Description:** Icon of a book, label "Description", input field "Enter description".

Right Screen: The header bar contains a profile picture placeholder on the left and the title "Add certificate" on the right. The main content area lists six fields, each with an icon and a label:

- Certificate Name:** Icon of a trophy, label "Certificate Name", input field "Enter certificate name".
- Student Name:** Icon of a person, label "Student Name", input field "Tran Van C".
- Student ID:** Icon of a document with a magnifying glass, label "Student ID", input field "52200003".
- Session:** Icon of a calendar, label "Session", input field "Enter session".
- Organization:** Icon of an envelope, label "Organization", input field "Enter organization".
- Description:** Icon of a book, label "Description", input field "Enter description".
- School:** Icon of a school building, label "School", input field "Enter school".

At the bottom of the right screen is a large, rounded rectangular button labeled "Add certificate".

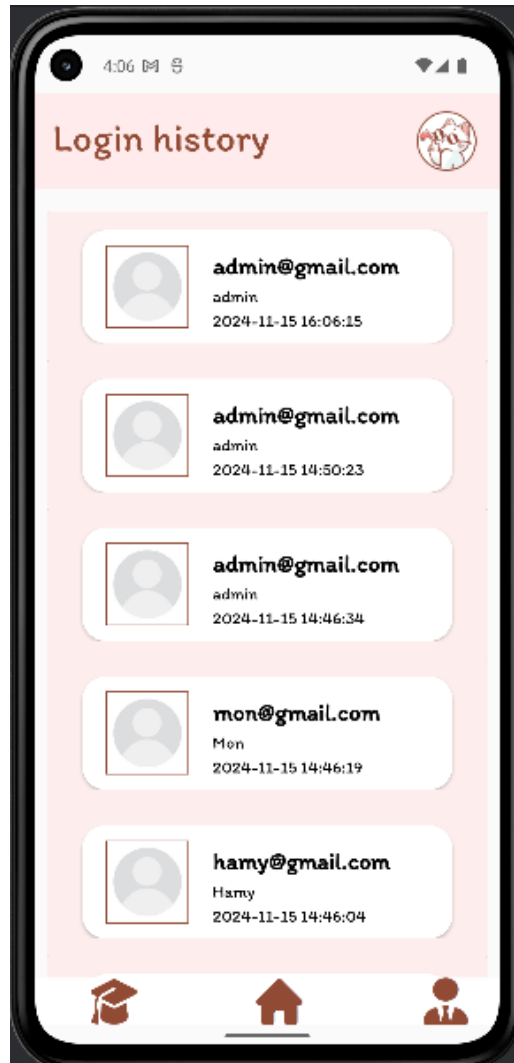
Hình 42: Màn hình thêm chứng chỉ cho sinh viên

Muốn thêm chứng chỉ cho sinh viên thì từ màn hình danh sách chứng chỉ của sinh viên đó bấm vào nút add certificate sẽ hiện ra trang thêm chứng chỉ như hai hình trên. Sau khi thêm các thông tin về chứng chỉ thì bấm nút add certificate để thực hiện thêm certificate vào firestore.



Hình 43: Màn hình xem và sửa thông tin certificate chi tiết của student

Tương tự như quản lý người dùng, khi bấm vào nút see more sẽ hiện ra trang thông tin chi tiết của chứng chỉ của sinh viên như hình bên trái. Khi bấm vào icon bánh răng ở góc trên bên phải của màn hình xem chi tiết thì sẽ hiện ra trang chỉnh sửa thông tin chi tiết certificate của sinh viên. Sau khi sửa thông tin và bấm nút Save thì thông tin sẽ được cập nhật và firestore.



Hình 44: Màn hình xem lịch sử đăng nhập

Khi bấm vào nút History trong màn hình quản lý user thì sẽ hiện thị màn hình như trên để xem được danh sách lịch sử đăng nhập của người dùng. Trên đó sẽ hiện email, tên người dùng và thời gian đăng nhập vào ứng dụng.

TÀI LIỆU THAM KHẢO

Tiếng Việt

1. <https://itviec.com/blog/firebase/>
2. https://topdev.vn/blog/firebase-la-gi/?utm_source=google&utm_medium=cpc&utm_campaign=pmax-branding&utm_content=performance&gad_source=1&gclid=Cj0KCQiAoae5BhCNARIsADV LZz dF3o0gBYRMvGkLsxz_LeWBjeHLXeY489fXI pOfJ2ND2Ugn76Pj u6gaAj92EALw_wcB#firebase-la-gi
3. <https://vinahost.vn/firebase-la-gi/#ftoc-heading-4>
4. <https://fptcloud.com/firebase-la-gi/>
5. <https://viblo.asia/p/tim-hieu-so-luoc-ve-firebase-Eb85oeOmZ2G>
6. <https://wiki.matbao.net/firebase-la-gi-giai-phap-lap-trinh-khong-can-backend-tu-google/#firebase-realtime-database-la-gi>
7. <https://firebase.google.com/docs/firestore/data-model?hl=vi>
8. <https://gcs.vn/cloud-firestore-la-gi/>

Tiếng Anh

9. <https://estuary.dev/firestore-vs-realtime-database/>